# pynssp Documentation

*Release 0.2.0*

**Gbedegnon Roseric Azondekon**

**Nov 03, 2026**

# Overview:


**1 About pynssp** **2**
- 1.1 Overview . . . 2
- 1.2 Installation . . . 2
- 1.3 Usage . . . 2
- 1.4 Contributing to this project . . . 3
- 1.5 Getting Help . . . 3
- 1.6 Public Domain Standard Notice . . . 4
- 1.7 License Standard Notice . . . 4
- 1.8 Privacy Standard Notice . . . 4
- 1.9 Contributing Standard Notice . . . 4
- 1.10 Records Management Standard Notice . . . 5

**2 Installing pynssp** **6**
- 2.1 Stable release . . . 6
- 2.2 Development version . . . 6
- 2.3 From sources . . . 6

**3 Getting started** **8**
- 3.1 Introduction . . . 8
- 3.2 Creating an NSSP user profile . . . 8
- 3.3 Time Series Data Table . . . 9
- 3.4 Time Series Graph from ESSENCE . . . 10
- 3.5 Table Builder Results . . . 11
- 3.6 Data Details (line-level) . . . 12
- 3.7 Summary Stats . . . 13
- 3.8 Alert List Detection Table . . . 13
- 3.9 Time Series data table with stratified, historical alerts (from ESSENCE2) . . . 14

**4 Anomaly Detection and Trend Classification** **15**
- 4.1 Introduction . . . 15
- 4.2 Data Pull from NSSP-ESSENCE . . . 16
- 4.3 Exponentially Weighted Moving Average (EWMA) . . . 16

4.4 Adaptive Multiple Regression . . . 18
4.5 Regression/EWMA Switch . . . 20
4.6 Farrington Temporal Detector . . . 22
4.7 Negative Binomial Regression . . . 24
4.8 Original Serfling Detector . . . 27
4.9 Trend Classification . . . 29

**5 Text Mining Example** **34**
5.1 Introduction . . . 34
5.2 Data Pull from NSSP-ESSENCE . . . 35
5.3 Tokenization . . . 37
5.4 n-gram Frequencies . . . 38
5.5 Chief Complaint Term Co-occurrence and Correlation . . . 39
5.6 Pearson Pairwise Correlation Chief Complaint Network Graph . . . 41

**6 pynssp** **46**
6.1 pynssp package . . . 46

**7 Release Notes** **65**
7.1 0.2.0 (2026-11-03) . . . 65
7.2 0.1.0 (2023-06-22) . . . 66

**8 Credits** **68**
8.1 Development Lead . . . 68
8.2 Contributors . . . 68

**9 Contributing** **69**
9.1 Welcome! . . . 69
9.2 Public Domain . . . 69
9.3 Requesting Changes . . . 69

**10 Disclaimer** **71**

**11 Creating a Culture of Innovation** **72**
11.1 Be Empowering . . . 72
11.2 Rules of Behavior . . . 72
11.3 Boundaries . . . 73
11.4 Background . . . 74
11.5 Credit . . . 74
11.6 Relevant Legal Considerations . . . 74

**12 Indices and tables** **75**

**Python Module Index** **76**

**Index** **77**


pynssp

Chapter 1

# About pynssp

## 1.1 Overview

pynssp is a Python package for the National Syndromic Surveillance Program (NSSP) and its Community of Practice. A collection of classes and methods to advance the practice of Syndromic Surveillance. It serves as a Python alternative to the Rnssp R package.

## 1.2 Installation

### 1.2.1 Stable release

To install pynssp, run this command in your terminal:

```
$ pip install pynssp
```

This is the preferred method to install pynssp, as it will always install the most recent stable release.

### 1.2.2 Development version

You can install the development version of pynssp from Github:

```
$ pip install git+https://github.com/CDCgov/pynssp.git
```

## 1.3 Usage

```
>>> from pynssp import *
>>> from datetime import date, timedelta
>>> import pandas as pd
```

(continues on next page)

(continued from previous page)

```
## Creating a user profile (token)
>>> myProfile = create_token_profile()

## Creating a user profile (username and password)
>>> myProfile = create_profile()

## JSON URL from NSSP-ESSENCE API
>>> url = "https://essence.syndromicsurveillance.org/nssp_essence/api/
 ↪alerts/regionSyndromeAlerts?end_date=31Jan2021&start_date=29Jan2021"

## Update Start and End dates in NSSP-ESSENCE API URL
>>> startDate = date.today() - timedelta(days=30)
>>> endDate = date.today()

>>> url = change_dates(url, start_date = startDate, end_date = endDate)

## Pull Time Series Data from NSSP-ESSENCE
>>> api_data = get_api_data(url, profile=myProfile)

## Inspect data object structure
>>> api_data.columns

## Extract table of interest
>>> api_data = pd.json_normalize(api_data["regionSyndromeAlerts"][0])

## Get a glimpse of the pulled dataset
>>> api_data.head()
```

## 1.4 Contributing to this project

Should you want to contribute to this project, submit a push request to this Github repository and consider submitting a request to be added as a developer to gazondekon@cdc.gov.

## 1.5 Getting Help

If you encounter a clear bug, please consider emailing the author at gazondekon@cdc.gov and/or file an issue with a minimal reproducible example.

## 1.6 Public Domain Standard Notice

This repository constitutes a work of the United States Government and is not subject to domestic copyright protection under 17 USC ? 105. This repository is in the public domain within the United States, and copyright and related rights in the work worldwide are waived through the CC0 1.0 Universal public domain dedication. All contributions to this repository will be released under the CC0 dedication. By submitting a pull request you are agreeing to comply with this waiver of copyright interest.

## 1.7 License Standard Notice

The repository utilizes code licensed under the terms of the Apache Software License and therefore is licensed under ASL v2 or later.

This source code in this repository is free: you can redistribute it and/or modify it under the terms of the Apache Software License version 2, or (at your option) any later version.

This source code in this repository is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the Apache Software License for more details.

You should have received a copy of the Apache Software License along with this program. If not, see http://www.apache.org/licenses/LICENSE-2.0.html

The source code forked from other open source projects will inherit its license.

## 1.8 Privacy Standard Notice

This repository contains only non-sensitive, publicly available data and information. All material and community participation is covered by the Disclaimer and Code of Conduct. For more information about CDC's privacy policy, please visit http://www.cdc.gov/other/privacy.html.

## 1.9 Contributing Standard Notice

Anyone is encouraged to contribute to the repository by forking and submitting a pull request. (If you are new to GitHub, you might start with a basic tutorial.) By contributing to this project, you grant a world-wide, royalty-free, perpetual, irrevocable, non-exclusive, transferable license to all users under the terms of the Apache Software License v2 or later.

All comments, messages, pull requests, and other submissions received through CDC including this GitHub page may be subject to applicable federal law, including but not limited to the Federal Records Act, and may be archived. Learn more at http://www.cdc.gov/other/privacy.html.

## 1.10 Records Management Standard Notice

This repository is not a source of government records, but is a copy to increase collaboration and collaborative potential. All government records will be published through the CDC web site.

Chapter 2

# Installing pynssp

## 2.1 Stable release

To install pynssp, run this command in your terminal:

```
$ pip install pynssp
```

This is the preferred method to install pynssp, as it will always install the most recent stable release.

If you don't have pip installed, this Python installation guide can guide you through the process.

## 2.2 Development version

You may install the development version with pip by running this command in your terminal.

```
$ pip install git+https://github.com/cdcgov/pynssp.git
```

If you don't have pip installed, this Python installation guide can guide you through the process.

## 2.3 From sources

Alternatively, the sources for pynssp can be downloaded from the Github repo.

You can either clone the public repository:

```
$ git clone git://github.com/cdcgov/pynssp
```

Or download the tarball:

```
$ curl -OJL https://github.com/cdcgov/pynssp/tarball/master
```

Once you have a copy of the source, you can install it with:

```
$ python setup.py install
```

Chapter 3

# Getting started

## 3.1 Introduction

The goal of the `pynssp` package is to facilitate access to the Electronic Surveillance System for the Early Notification of Community-based Epidemics (ESSENCE) via a secure and simplified interface. In addition, `pynssp` provides methods and functions that streamline data retrieval by abstracting the complexity of Python scripts from users.

In this vignette, we explain how to create an NSSP user profile and provide examples of how to use it to pull data from ESSENCE using the following ESSENCE APIs:

- Time Series data table
- Time Series PNG image
- Table Builder Results
- Data Details (line-level)
- Summary stats
- Alert list detection table
- Time Series data table with stratified, historical alerts (from ESSENCE2)

## 3.2 Creating an NSSP user profile

We start by loading the `pandas` and the `pynssp` packages.

```
>>> import pandas as pd
>>> from pynssp.utils import *
```

The next step is to create an NSSP user profile by creating an object of the `Credentials` class. Here, we use the `create_profile()` function to create a user profile.

```
>>> myProfile = create_profile()

# Save profile object to file for future use
>>> myProfile.pickle()

# Load profile object
# myProfile = pd.read_pickle("myProfile.pkl")
```

The above code needs to be executed only once. Upon execution, it prompts the user to provide their username and password.

The created `myProfile` object includes the `.get_api_response()`, `.get_api_data()`, and `.get_api_graph()` methods, each with parameters for pulling ESSENCE data. Alternatively, the `get_api_response()`, `get_api_data()`, and `get_api_graph()` functions serve as wrappers for their respective methods. The `get_essence_data()` function may be used for NSSP-ESSENCE API URLs.

In the following sections, we show how to pull data from ESSENCE using the seven APIs listed above.

## 3.3 Time Series Data Table

```
>>> url = "https://essence.syndromicsurveillance.org/nssp_essence/api/
↪timeSeries?endDate=9Feb2021&medicalGrouping=injury&
↪percentParam=noPercent&geographySystem=hospitaldhhsregion&datasource=va_
↪hospdreg&detector=probrepswitch&startDate=11Nov2020&
↪timeResolution=daily&medicalGroupingSystem=essencesyndromes&userId=455&
↪aqtTarget=TimeSeries"

# Pull time series data
>>> api_data_ts = get_api_data(url, profile=myProfile) # or api_data_ts =␣
↪myProfile.get_api_data(url)

>>> api_data_ts.columns

# Extracting embedded dataframe
>>> api_data_ts = pd.json_normalize(api_data_ts["timeSeriesData"][0])

# Preview data
>>> api_data_ts.head()
```

Alternatively, the example below uses the `get_essence_data()` function to achieve the same outcome and extract the embedded dataframe directly when needed.

```
>>> api_data_ts = get_essence_data(url, profile=myProfile)

# Preview data
>>> api_data_ts.head()
```

## 3.4 Time Series Graph from ESSENCE

The example below shows how to retrieve the Time Series Graph from ESSENCE and insert it into a Jupyter notebook.

```
>>> url = "https://essence.syndromicsurveillance.org/nssp_essence/api/
 ↪timeSeries/graph?endDate=9Feb2021&medicalGrouping=injury&
 ↪percentParam=noPercent&geographySystem=hospitaldhhsregion&datasource=va_
 ↪hospdreg&detector=probrepswitch&startDate=11Nov2020&
 ↪timeResolution=daily&medicalGroupingSystem=essencesyndromes&userId=455&
 ↪aqtTarget=TimeSeries&graphTitle=National%20-%20Injury%20Syndrome%20Daily
 ↪%20Counts&xAxisLabel=Date&yAxisLabel=Count"

# Data pull from ESSENCE
>>> api_data_graph = get_api_graph(url, profile=myProfile)

# Check the type of api_data_graph
>>> type(api_data_graph)

# Print image file location
>>> print(api_data_graph)

# Insert it into a Jupyter notebook
>>> api_data_graph.plot()
```

From the example above, the variable `api_data_graph` is an `APIGraph` object. In an interactive mode from the Python console, the `.show()` method can be called on the `APIGraph` object to preview the image it contains.

```
>>> api_data_graph.show()
```

For an exhaustive list of methods that may be called on an `APIGraph` object, please see the `pynssp` documentation.

## 3.5 Table Builder Results

The CSV option of the Table Builder Results API pulls in data in the tabular format seen on ESSENCE. The JSON option on the other hand, pulls in the data in a long, pivoted format. In the following subsections, we demonstrate how to pull the Table Builder results data with both options.

### 3.5.1 CSV option

```
>>> url = "https://essence.syndromicsurveillance.org/nssp_essence/api/
↪tableBuilder/csv?endDate=31Dec2020&ccddCategory=cdc%20opioid%20overdose
↪%20v3&percentParam=noPercent&geographySystem=hospitaldhhsregion&
↪datasource=va_hospdreg&detector=nodetectordetector&startDate=1Oct2020&
↪ageNCHS=11-14&ageNCHS=15-24&ageNCHS=25-34&ageNCHS=35-44&ageNCHS=45-54&
↪ageNCHS=55-64&ageNCHS=65-74&ageNCHS=75-84&ageNCHS=85-1000&
↪ageNCHS=unknown&timeResolution=monthly&hasBeenE=1&
↪medicalGroupingSystem=essencesyndromes&userId=455&
↪aqtTarget=TableBuilder&rowFields=timeResolution&
↪rowFields=geographyhospitaldhhsregion&columnField=ageNCHS"

# Data pull from ESSENCE
>>> api_data_tb_csv = get_api_data(url, fromCSV=True, profile=myProfile)

# Preview data
>>> api_data_tb_csv.head()
```

### 3.5.2 JSON option

While the `get_api_data()` function can also be used to pull data from ESSENCE API URLs returning JSON objects, the `get_essence_data()` function has the advantage of not requiring extra JSON parsing, as described in the second section of this tutorial.

```
>>> url = "https://essence2.syndromicsurveillance.org/nssp_essence/api/
↪tableBuilder?endDate=31Dec2020&ccddCategory=cdc%20opioid%20overdose
↪%20v3&percentParam=noPercent&geographySystem=hospitaldhhsregion&
↪datasource=va_hospdreg&detector=nodetectordetector&startDate=1Oct2020&
↪ageNCHS=11-14&ageNCHS=15-24&ageNCHS=25-34&ageNCHS=35-44&ageNCHS=45-54&
↪ageNCHS=55-64&ageNCHS=65-74&ageNCHS=75-84&ageNCHS=85-1000&
↪ageNCHS=unknown&timeResolution=monthly&hasBeenE=1&
↪medicalGroupingSystem=essencesyndromes&userId=2362&
↪aqtTarget=TableBuilder&rowFields=timeResolution&
↪rowFields=geographyhospitaldhhsregion&columnField=ageNCHS"
```

(continues on next page)

(continued from previous page)

```
# Data pull from ESSENCE
>>> api_data_tb_json = get_essence_data(url, profile=myProfile)

# Preview data
>>> api_data_tb_json.head()
```

## 3.6 Data Details (line-level)

Similar to the Table Builder Results API, the Data Details (line-level) API provides CSV and JSON data outputs.

### 3.6.1 CSV option

```
>>> url = "https://essence.syndromicsurveillance.org/nssp_essence/api/
 ↪dataDetails/csv?medicalGrouping=injury&geography=region%20i&
 ↪percentParam=noPercent&geographySystem=hospitaldhhsregion&datasource=va_
 ↪hospdreg&detector=probrepswitch&timeResolution=daily&
 ↪medicalGroupingSystem=essencesyndromes&userId=455&aqtTarget=TimeSeries&
 ↪startDate=31Jan2021&endDate=31Jan2021"

# Data pull from ESSENCE
>>> api_data_dd_csv = get_api_data(url, fromCSV=True, profile=myProfile)

# Preview data
>>> api_data_dd_csv.head()
```

### 3.6.2 JSON option

```
>>> url = "https://essence.syndromicsurveillance.org/nssp_essence/api/
 ↪dataDetails?endDate=31Jan2021&medicalGrouping=injury&
 ↪percentParam=noPercent&geographySystem=hospitaldhhsregion&datasource=va_
 ↪hospdreg&detector=probrepswitch&startDate=31Jan2021&
 ↪timeResolution=daily&medicalGroupingSystem=essencesyndromes&userId=455&
 ↪aqtTarget=DataDetails"

# Data pull from ESSENCE
>>> api_data_dd_json = get_essence_data(url, profile=myProfile)

# Preview data
>>> api_data_dd_json.head()
```

## 3.7 Summary Stats

```
>>> url = "https://essence.syndromicsurveillance.org/nssp_essence/api/
 ↪summaryData?endDate=31Jan2021&medicalGrouping=injury&geography=region
 ↪%20i&percentParam=noPercent&geographySystem=hospitaldhhsregion&
 ↪datasource=va_hosp&detector=probrepswitch&startDate=29Jan2021&
 ↪timeResolution=daily&medicalGroupingSystem=essencesyndromes&userId=455&
 ↪aqtTarget=TimeSeries"

# Data pull from ESSENCE
>>> api_data_ss = get_essence_data(url, profile=myProfile)

# Preview data
>>> api_data_ss.head()
```

## 3.8 Alert List Detection Table

Since the Alert List API provides programmatic access to the Alert List table in the ESSENCE user interface by patient region or hospital region, we provide two data pull use cases in the following subsections:

### 3.8.1 Alert List Detection Table by Patient Region

```
>>> url = "https://essence.syndromicsurveillance.org/nssp_essence/api/
 ↪alerts/regionSyndromeAlerts?end_date=31Jan2021&start_date=29Jan2021"

# Data pull from ESSENCE
>>> api_data_alr = get_essence_data(url, profile=myProfile)

# Preview data
>>> api_data_alr.head()
```

### 3.8.2 Alert List Detection Table by Hospital Region

```
>>> url = "https://essence.syndromicsurveillance.org/nssp_essence/api/
 ↪alerts/hospitalSyndromeAlerts?end_date=31Jan2021&start_date=29Jan2021"

# Data pull from ESSENCE
>>> api_data_alh = get_api_data(url, profile=myProfile)

# Preview data
>>> api_data_alh.head()
```

## 3.9 Time Series data table with stratified, historical alerts (from ESSENCE2)

As of February 10, 2023, this functionality is available from ESSENCE2. Therefore, if your ESSENCE 2 credentials are different from the ones defined above for ESSENCE, you will need to create another profile object for ESSENCE 2 and use it to run the code below. In this example, we assume the same user profile is used for both ESSENCE and ESSENCE 2.

```
>>> url = "https://essence2.syndromicsurveillance.org/nssp_essence/api/
 ↪timeSeries?endDate=9Feb2021&ccddCategory=cdc%20pneumonia%20ccdd%20v1&
 ↪ccddCategory=cdc%20coronavirus-dd%20v1&ccddCategory=cli%20cc%20with
 ↪%20cli%20dd%20and%20coronavirus%20dd%20v2&percentParam=ccddCategory&
 ↪geographySystem=hospitaldhhsregion&datasource=va_hospdreg&
 ↪detector=probrepswitch&startDate=11Nov2020&timeResolution=daily&
 ↪hasBeenE=1&medicalGroupingSystem=essencesyndromes&userId=2362&
 ↪aqtTarget=TimeSeries&stratVal=ccddCategory&multiStratVal=geography&
 ↪graphOnly=true&numSeries=3&graphOptions=multipleSmall&
 ↪seriesPerYear=false&nonZeroComposite=false&removeZeroSeries=true&
 ↪startMonth=January&stratVal=ccddCategory&multiStratVal=geography&
 ↪graphOnly=true&numSeries=3&graphOptions=multipleSmall&
 ↪seriesPerYear=false&startMonth=January&nonZeroComposite=false"

# Data pull from ESSENCE
>>> api_data_tssh = get_essence_data(url, profile=myProfile)

# Preview data
>>> api_data_tssh.head()
```

Chapter 4

# Anomaly Detection and Trend Classification

## 4.1 Introduction

In this tutorial, we describe how to perform anomaly detection and trend classification analysis using time series data from NSSP-ESSENCE.

This tutorial uses data from the NSSP-ESSENCE source for the CLI CC with CLI DD and Coronavirus DD v2 definition, limited to ED visits (Has been Emergency = "Yes").

We start this tutorial by loading the `pynssp` package.

```
>>> from pynssp import *
```

Then, we load additional packages for visualization purposes:

```
>>> import pandas as pd
>>> import seaborn as sns
>>> import matplotlib.pyplot as plt
>>> import matplotlib.dates as mdates
>>> from matplotlib.patches import Patch
```

Next, we create an NSSP user profile by creating an object of the `Credentials` class.

```
>>> myProfile = create_profile() # Creating an ESSENCE user profile

# Save profile object to file for future use
>>> myProfile.pickle()
```

## 4.2 Data Pull from NSSP-ESSENCE

With the `myProfile` object, we authenticate to NSSP-ESSENCE and pull data using the Time Series data table API.

```
>>> url = "https://essence2.syndromicsurveillance.org/nssp_essence/api/
 ↪timeSeries?endDate=20Nov20&ccddCategory=cli%20cc%20with%20cli%20dd%20and
 ↪%20coronavirus%20dd%20v2&percentParam=ccddCategory&
 ↪geographySystem=hospitaldhhsregion&datasource=va_hospdreg&
 ↪detector=probrepswitch&startDate=22Aug20&timeResolution=daily&
 ↪hasBeenE=1&medicalGroupingSystem=essencesyndromes&userId=2362&
 ↪aqtTarget=TimeSeries&stratVal=&multiStratVal=geography&graphOnly=true&
 ↪numSeries=0&graphOptions=multipleSmall&seriesPerYear=false&
 ↪nonZeroComposite=false&removeZeroSeries=true&startMonth=January&
 ↪stratVal=&multiStratVal=geography&graphOnly=true&numSeries=0&
 ↪graphOptions=multipleSmall&seriesPerYear=false&startMonth=January&
 ↪nonZeroComposite=false"

# Data Pull from NSSP-ESSENCE
>>> api_data = get_essence_data(url, profile = myProfile)

# Inspect pulled data frame
>>> api_data.info()
```

Before applying an anomaly detection function, let's first group the data by HHS regions:

```
>>> df_hhs = api_data.groupby("hospitaldhhsregion_display")
```

## 4.3 Exponentially Weighted Moving Average (EWMA)

The Exponentially Weighted Moving Average (EWMA) compares a weighted average of the most recent visit counts to a baseline expectation. For the weighted average to be tested, an exponential weighting gives the most influence to the most recent observations.

This algorithm is appropriate for daily counts that do not have the characteristic features modeled in the regression algorithm. It is more applicable for Emergency Department data from certain hospital groups and for time series with small counts (daily average below 10) because of the limited case definition or chosen geographic region.

The EWMA detection algorithm can be performed with `alert_ewma()` function (run `help(alert_ewma)` in your Python console or `alert_ewma?` in Jupyter Notebook or JupyterLab for more).

```
>>> df_ewma = alert_ewma(df_hhs, t = "date", y = "dataCount")
```

Let's subset the dataframe to visualize the time series with the anomalies for Region 4:

```
>>> df_ewma_region = df_ewma[df_ewma["hospitaldhhsregion_display"] ==
↪"Region 4"]
```

Now, let's visualize the time series with the anomalies

```
>>> sns.set_style('whitegrid') # Set plot grid
>>> sns.set_style("white", {"axes.grid": False})

# Set plot size and y-axis limit
>>> plt.figure(figsize=(12, 6))
>>> plt.gca().set_ylim([0, 5000])

# Plot data
>>> sns.lineplot(data=df_ewma_region, x='date', y='dataCount', color='blue
↪', linewidth=1)

>>> sns.scatterplot(
...     data=df_ewma_region, x='date', y='dataCount', hue='alert',
...     palette=['black', 'blue', 'darkorange', 'red'], alpha=1
... )

# Format dates
>>> plt.gca().yaxis.set_major_formatter(
...     plt.FuncFormatter(lambda x, loc: "{:,}".format(int(x)))
... )
>>> plt.gca().xaxis.set_major_locator(mdates.WeekdayLocator())
>>> plt.gcf().autofmt_xdate()

# Set x and y axes labels
>>> plt.gca().set_xlabel('Date')
>>> plt.gca().set_ylabel('Count')

# Customize legend
>>> legend = plt.legend(
...     title='Detection', loc='lower center', bbox_to_anchor=(0.5, -0.5),
...     ncol=5, frameon=True, framealpha=0.5, edgecolor='black',␣
↪borderpad=0.75
... )

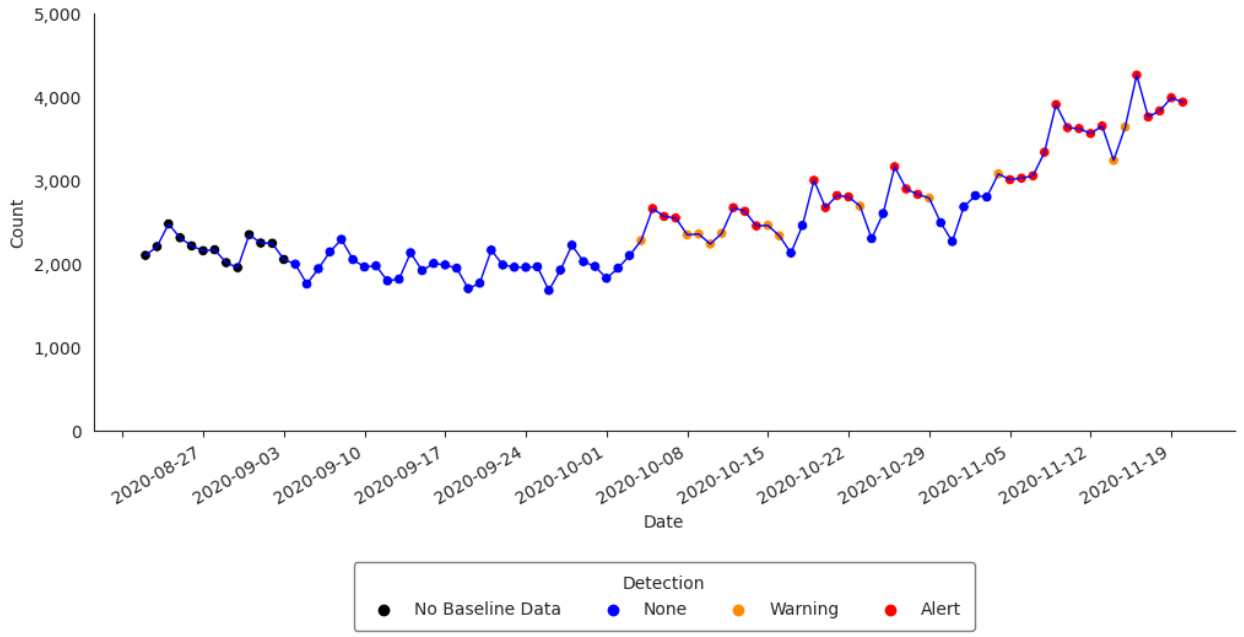
>>> legend.get_texts()[0].set_text('No Baseline Data')
>>> legend.get_texts()[1].set_text('None')
>>> legend.get_texts()[2].set_text('Warning')
```

(continues on next page)

(continued from previous page)

```
>>> legend.get_texts()[3].set_text('Alert')

# Remove the top and right spines from plot
>>> sns.despine()
```




## 4.4 Adaptive Multiple Regression

The Adaptive Multiple Regression algorithm fits a linear model to a baseline of counts or percentages and forecasts a predicted value for test dates following a predefined buffer period.

This model includes terms to account for linear trends and day-of-week effects. This implementation does NOT include holiday terms as in the Regression 1.4 algorithm in ESSENCE.

The Adaptive Multiple Regression detector can be performed with the `alert_regression()` function (run `help(alert_regression)` in your Python console or `alert_regression?` in Jupyter Notebook or JupyterLab for more).

```
>>> df_regression = alert_regression(df_hhs, t = "date", y = "dataCount")
```

Let's filter `df_regression` and visualize the time series with the anomalies for Region 4:

```
>>> df_regression_region = df_regression[df_regression[
↪"hospitaldhhsregion_display"] == "Region 4"]
```

```
>>> sns.set_style('whitegrid') # Set plot grid
>>> sns.set_style("white", {"axes.grid": False})

# Set plot size and y-axis limit
>>> plt.figure(figsize=(12, 6))
>>> plt.gca().set_ylim([0, 5000])

# Plot data
>>> sns.lineplot(data=df_regression_region, x='date', y='dataCount',␣
↪color='blue', linewidth=1)

>>> sns.scatterplot(
...     data=df_regression_region, x='date', y='dataCount', hue='alert',
...     palette=['black', 'blue', 'darkorange', 'red'], alpha=1
... )

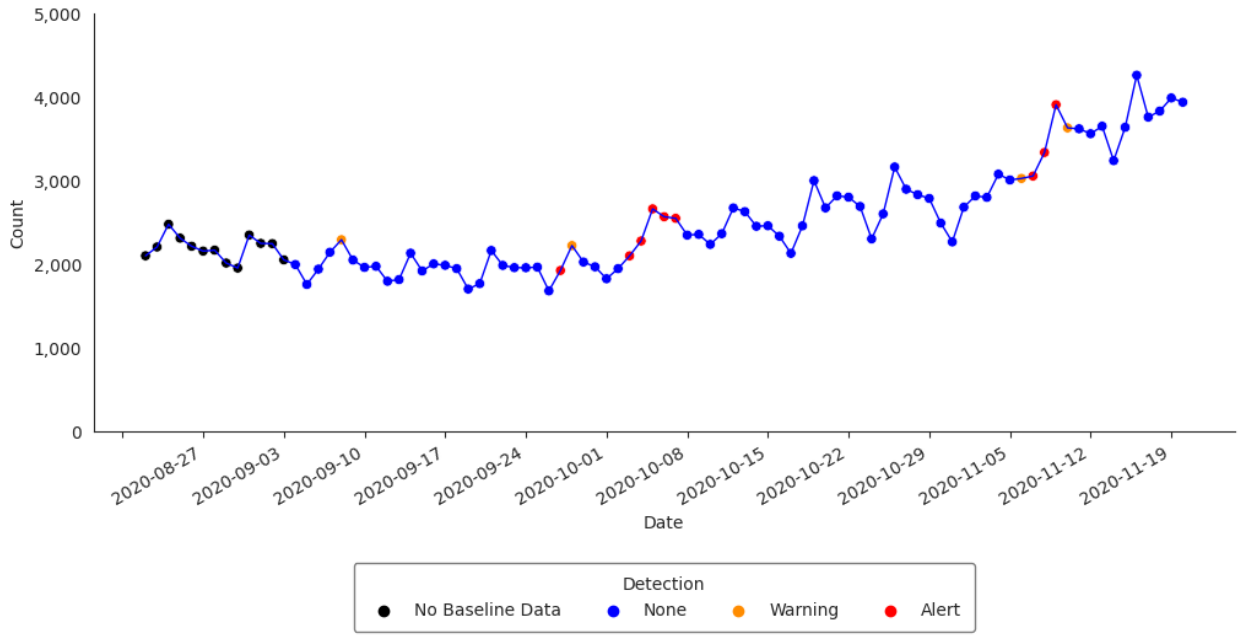
# Format dates
>>> plt.gca().yaxis.set_major_formatter(
...     plt.FuncFormatter(lambda x, loc: "{:,}".format(int(x)))
... )
>>> plt.gca().xaxis.set_major_locator(mdates.WeekdayLocator())
>>> plt.gcf().autofmt_xdate()

# Set x and y axes labels
>>> plt.gca().set_xlabel('Date')
>>> plt.gca().set_ylabel('Count')

# Customize legend
>>> legend = plt.legend(
...     title='Detection', loc='lower center', bbox_to_anchor=(0.5, -0.5),
...     ncol=5, frameon=True, framealpha=0.5, edgecolor='black',␣
↪borderpad=0.75
... )

>>> legend.get_texts()[0].set_text('No Baseline Data')
>>> legend.get_texts()[1].set_text('None')
>>> legend.get_texts()[2].set_text('Warning')
>>> legend.get_texts()[3].set_text('Alert')

# Remove the top and right spines from plot
>>> sns.despine()
```




## 4.5 Regression/EWMA Switch

The NSSP-ESSENCE Regression/EWMA Switch algorithm generalizes the Regression and EWMA algorithms by applying the most appropriate algorithm for baseline data.

First, adaptive multiple regression is applied and the adjusted R-squared value of the model is examined to see if it meets a threshold of >= 0.60. If this threshold is not met, then the model is considered to not explain the data well. In this case, the algorithm switches to the EWMA algorithm, which is more appropriate for sparser time series that are common with granular geographic levels.

The Regression/EWMA algorithm can be performed with the `alert_switch()` function (run `help(alert_switch)` in your Python console or `alert_switch?` in Jupyter Notebook or JupyterLab for more).

```
>>> df_switch = alert_switch(df_hhs, t = "date", y = "dataCount")
```

Let's visualize the time series with the anomalies for Region 4:

```
>>> df_switch_region = df_switch[df_switch["hospitaldhhsregion_display"]␣
↪== "Region 4"]
```

```
>>> sns.set_style('whitegrid') # Set plot grid
>>> sns.set_style("white", {"axes.grid": False})

# Set plot size and y-axis limit
>>> plt.figure(figsize=(12, 6))
```

(continues on next page)

(continued from previous page)

```
>>> plt.gca().set_ylim([0, 5000])

# Plot data
>>> sns.lineplot(data=df_switch_region, x='date', y='dataCount', color=
↪'blue', linewidth=1)

>>> sns.scatterplot(
...     data=df_switch_region, x='date', y='dataCount', hue='alert',
...     palette=['black', 'blue', 'darkorange', 'red'], alpha=1
... )

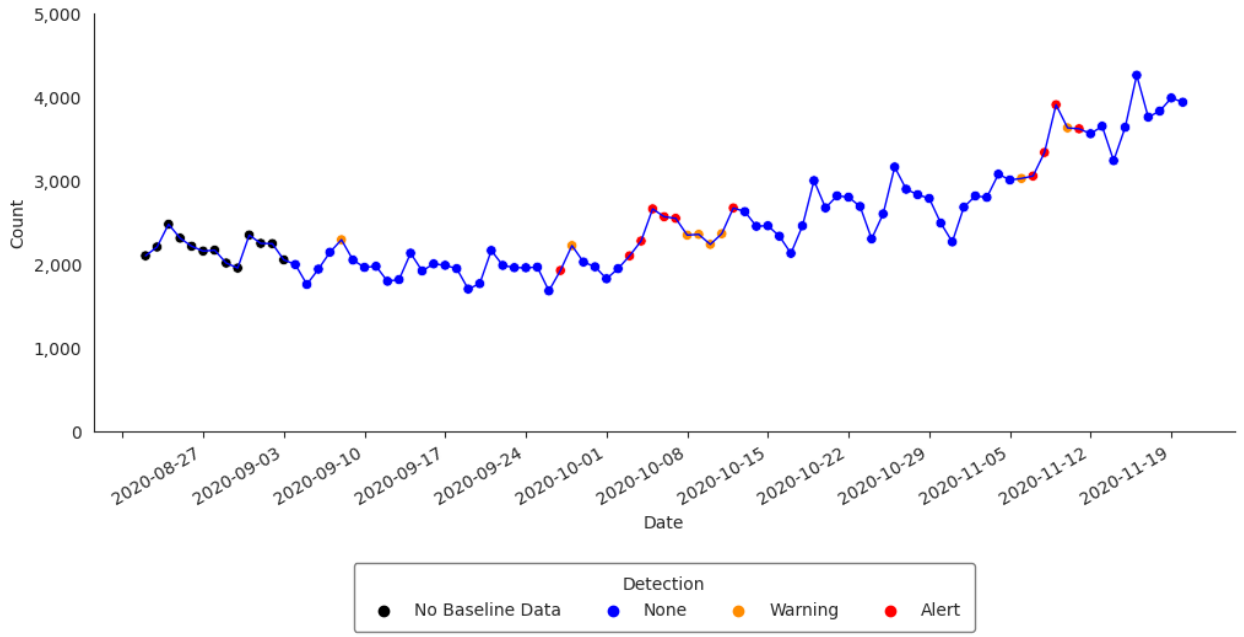
# Format dates
>>> plt.gca().yaxis.set_major_formatter(
...     plt.FuncFormatter(lambda x, loc: "{:,}".format(int(x)))
... )
>>> plt.gca().xaxis.set_major_locator(mdates.WeekdayLocator())
>>> plt.gcf().autofmt_xdate()

# Set x and y axes labels
>>> plt.gca().set_xlabel('Date')
>>> plt.gca().set_ylabel('Count')

# Customize legend
>>> legend = plt.legend(
...     title='Detection', loc='lower center', bbox_to_anchor=(0.5, -0.5),
...     ncol=5, frameon=True, framealpha=0.5, edgecolor='black',␣
↪borderpad=0.75
... )

>>> legend.get_texts()[0].set_text('No Baseline Data')
>>> legend.get_texts()[1].set_text('None')
>>> legend.get_texts()[2].set_text('Warning')
>>> legend.get_texts()[3].set_text('Alert')

# Remove the top and right spines from plot
>>> sns.despine()
```




## 4.6 Farrington Temporal Detector

The Farrington algorithm is intended for weekly time series of counts spanning multiple years.

The Original Farrington Algorithm uses quasi-Poisson generalized linear regression models that are fit to baseline counts associated with reference dates in a defined number of previous years, including a defined number of weeks before and after each reference date.

The Modified implementation of the original Farrington algorithm improves performance by including more historical data in the baseline.

Both the Original and the Modified Farrington algorithms can be performed with the `alert_farrington()` function. By default, `alert_farrington()` implements the original Farrington. Add `method = "modified"` to the function to apply the modified Farrington algorithm to your data (run `help(alert_farrington)` in the Python console or `alert_farrington?` in Jupyter Notebook or JupyterLab for more).

The example below applies the Farrington detectors to a synthetic time series prepackaged in `pynssp`. The following example uses the Scenario #1 time series with moderate counts, 4 injected outbreaks over weeks 1-261, and 1 injected outbreak during weeks 262-313. The methods outlined in "An Improved Algorithm for Outbreak Detection in Multiple Surveillance Systems" (Angela Noufaily et al. [2012]) were used to generate these synthetic time series and inject outbreak cases.

The Original Farrington temporal detector can be applied as below:

```
>>> synth_ts1 = get_scenario1() # Load synthesized time series data for␣
↪Scenario #1
>>> synth_ts1.head()
```

Now, applying the Original Farrington detector:

```
>>> df_farr_original = alert_farrington(synth_ts1, t = "date", y = "cases
↪")
```

Similarly, the Modified Farrington temporal detector can be applied as below:

```
>>> df_farr_modified = alert_farrington(
...     synth_ts1,
...     t = "date",
...     y = "cases",
...     method = "modified"
... )
```

Let's visualize the time series with the detected anomalies:

```
>>> fig, ax = plt.subplots(figsize=(12, 6))

>>> sns.lineplot(
...     data=df_farr_modified, x='date', y='cases',
...     color='#1A476F', label='Observed', ax=ax
... )

>>> sns.scatterplot(
...     data=df_farr_modified[df_farr_modified['alert'] == 'red'],
...     x='date', y='cases', color='#A50026', ax=ax
... )

# Set axis labels and limits
>>> ax.set_xlabel('Date')
>>> ax.set_ylabel('Count')
>>> ax.set_ylim(bottom=0)
>>> ax.yaxis.set_major_formatter(
...     plt.FuncFormatter(lambda x, loc: "{:,}".format(int(x)))
... )

# Set legend
>>> ax.legend(
...     title="Series", loc='lower center', bbox_to_anchor=(0.5, -0.3),
...     ncol=2, framealpha=1, edgecolor='black'
... )

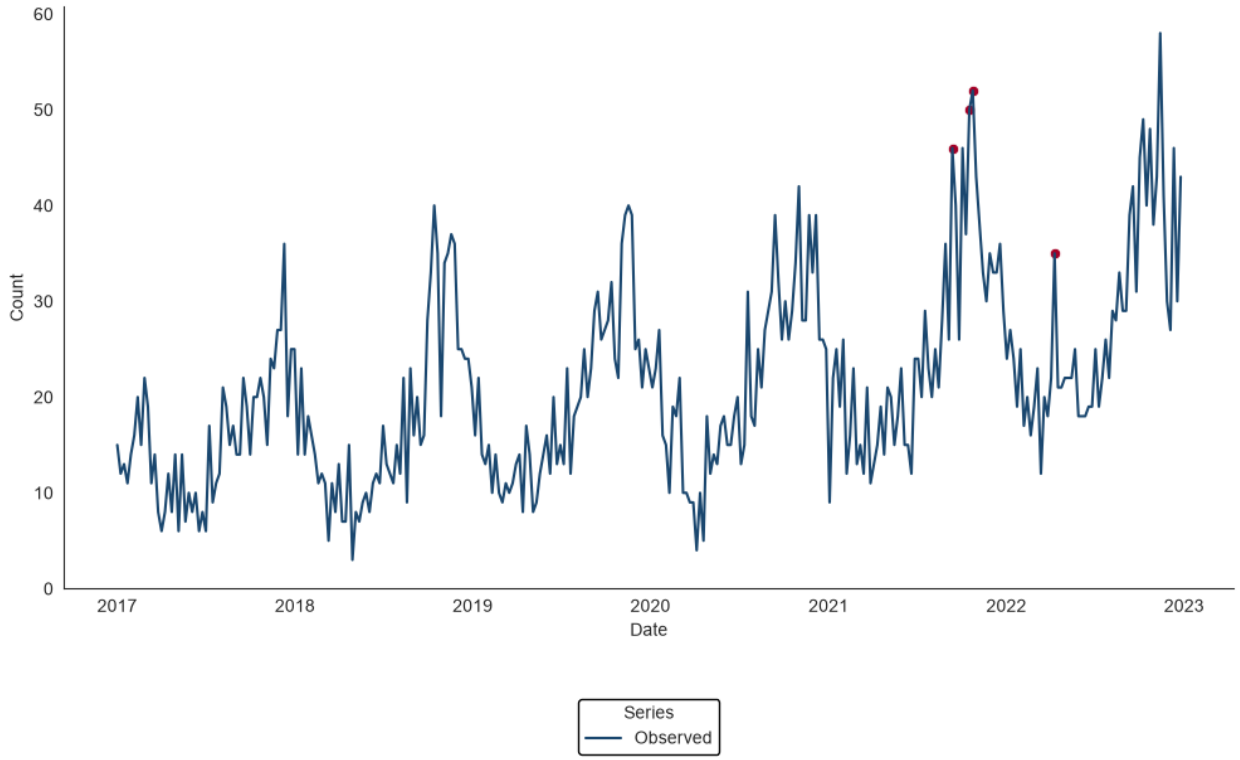
# Set date axis
>>> ax.xaxis.set_major_locator(mdates.YearLocator())
>>> ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
```

(continues on next page)

(continued from previous page)

```
# Set tick parameters
>>> ax.tick_params(axis='both', length=2.5, width=0.5)

# Set theme
>>> sns.set_style('white')
>>> sns.despine()
```




## 4.7 Negative Binomial Regression

The Negative Binomial Regression algorithm is intended for weekly time series spanning multiple years and fits a negative binomial regression model with a time term and cyclic sine and cosine terms to a baseline period that spans 2 or more years.

Inclusion of cyclic terms in the model is intended to account for seasonality common in multi-year weekly time series of counts for syndromes and diseases such as influenza, RSV, and norovirus. Each baseline model is used to make weekly forecasts for all weeks following the baseline period. One-sided upper 95% prediction interval bounds are computed for each week in the prediction period. Alarms are signaled for any week during which the observed weekly count exceeds the upper bound of the prediction interval.

The Negative Binomial Regression detector can be applied with the `alert_nbinom()` function (run `help(alert_nbinom)` in your Python console or `alert_nbinom?` in Jupyter Notebook or

JupyterLab for more).

The example below applies the Negative Binomial Regression detector to our synthetic time series for Scenario #1.

```
>>> synth_ts1 = get_scenario1() # Load synthesized time series data for␣
↪Scenario #1
>>> synth_ts1.head()
```

Applying the Negative Binomial detector…

```
>>> df_nbinom = alert_nbinom(synth_ts1, t='date', y='cases', baseline_end=
↪'2021-12-26')
```

```
>>> fig, ax = plt.subplots(figsize=(12, 6))

>>> sns.lineplot(
...     data=df_nbinom, x='date', y='cases',
...     color='#1A476F', label='Observed', ax=ax
... )

>>> sns.lineplot(
...     data=df_nbinom, x='date', y='threshold',
...     color='#A50026', linewidth=0.5, label='Threshold', ax=ax
... )

>>> sns.scatterplot(
...     data=df_nbinom.loc[df_nbinom['alarm']],
...     x='date', y='cases', color='#A50026', ax=ax
... )

# Add vertical line
>>> ax.axvline(x=pd.to_datetime('2021-12-26'), color='black', linewidth=0.
↪2)

# Add text annotation
>>> ax.text(pd.to_datetime('2019-12-01'), 60, 'Baseline Data', ha='left',␣
↪va='center')

# Set axis labels and limits
>>> ax.set_xlabel('Date')
>>> ax.set_ylabel('Count')
>>> ax.set_ylim(bottom=0)
>>> ax.yaxis.set_major_formatter(
...     plt.FuncFormatter(lambda x, loc: "{:,}".format(int(x)))
```

(continues on next page)

(continued from previous page)

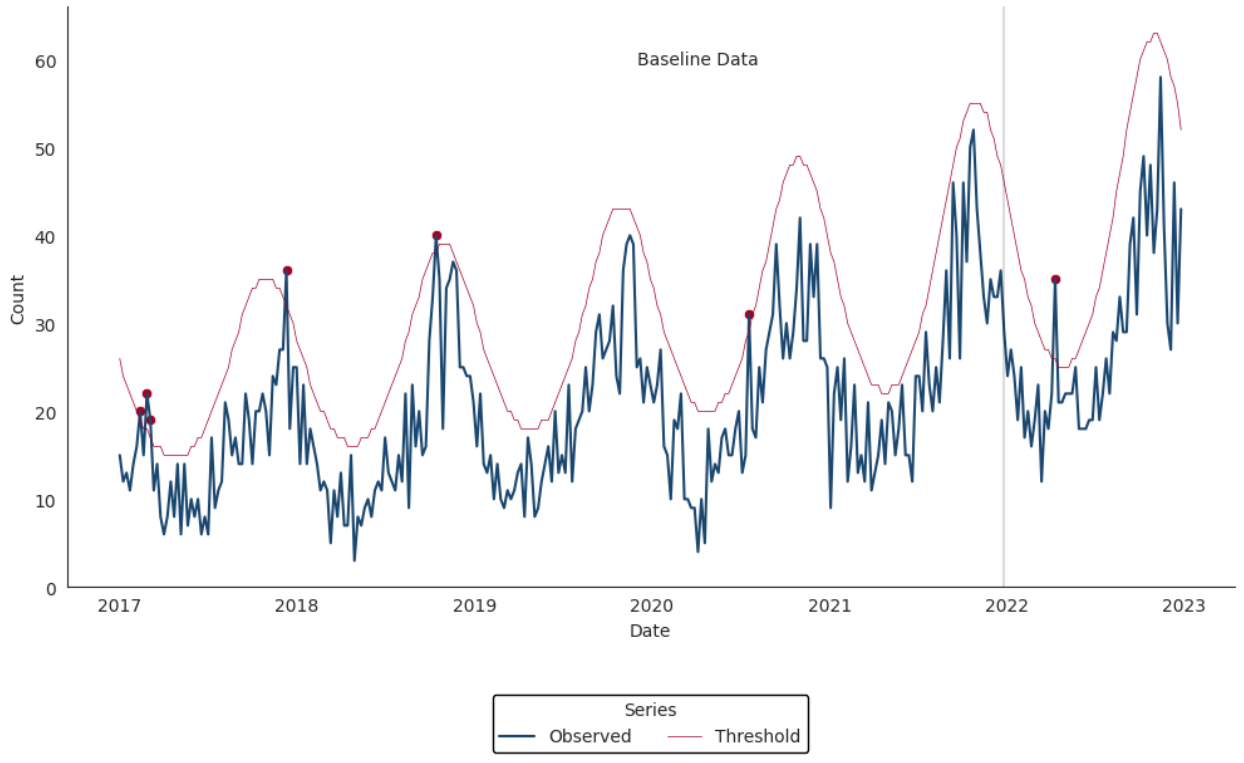
```
... )

# Set legend
>>> ax.legend(
...     title="Series", loc='lower center', bbox_to_anchor=(0.5, -0.3),
...     ncol=2, framealpha=1, edgecolor='black'
... )

# Set date axis
>>> ax.xaxis.set_major_locator(mdates.YearLocator())
>>> ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))

# Set tick parameters
>>> ax.tick_params(axis='both', length=2.5, width=0.5)

# Set theme
>>> sns.set_style('white')
>>> sns.despine()
```



## 4.8 Original Serfling Detector

The Original Serfling detector is intended for weekly time series spanning multiple years. It fits a linear regression model with a time term and sine and cosine terms to a baseline period that ideally spans 5 or more years.

Inclusion of Fourier terms in the model is intended to account for seasonality common in multi-year weekly time series. This implementation follows the approach of the original Serfling method in which weeks between October of the starting year of a season and May of the ending year of a season are considered to be in the epidemic period. Weeks in the epidemic period are removed from the baseline prior to fitting the regression model. Each baseline model is used to make weekly forecasts for all weeks following the baseline period. One-sided upper 95% prediction interval bounds are computed for each week in the prediction period. Alarms are signaled for any week during which the observed weekly count exceeds the upper bound of the prediction interval.

The Original Serfling detector can be applied with the `alert_serfling()` function (run `help(alert_serfling)` in your Python console or `alert_serfling?` in Jupyter Notebook or JupyterLab for more).

Using the same simulated time series from the previous examples, the Original Serfling detector can be applied as below:

```
>>> df_serfling  = alert_serfling(synth_ts1, t='date', y='cases',␣
↪baseline_end='2021-12-26')
```

```
>>> fig, ax = plt.subplots(figsize=(12, 6))

>>> sns.lineplot(
...     data=df_serfling, x='date', y='cases',
...     color='#1A476F', label='Observed', ax=ax
... )

>>> sns.lineplot(
...     data=df_serfling, x='date', y='threshold',
...     color='#A50026', linewidth=0.5, label='Threshold', ax=ax
... )

>>> sns.scatterplot(
...     data=df_serfling.loc[df_serfling['alarm']],
...     x='date', y='cases', color='#A50026', ax=ax
... )

# Add vertical line
>>> ax.axvline(x=pd.to_datetime('2021-12-26'), color='black', linewidth=0.
↪2)
```

(continues on next page)

(continued from previous page)

```
# Add text annotation
>>> ax.text(pd.to_datetime('2019-12-01'), 60, 'Baseline Data', ha='left',␣
 ↪va='center')

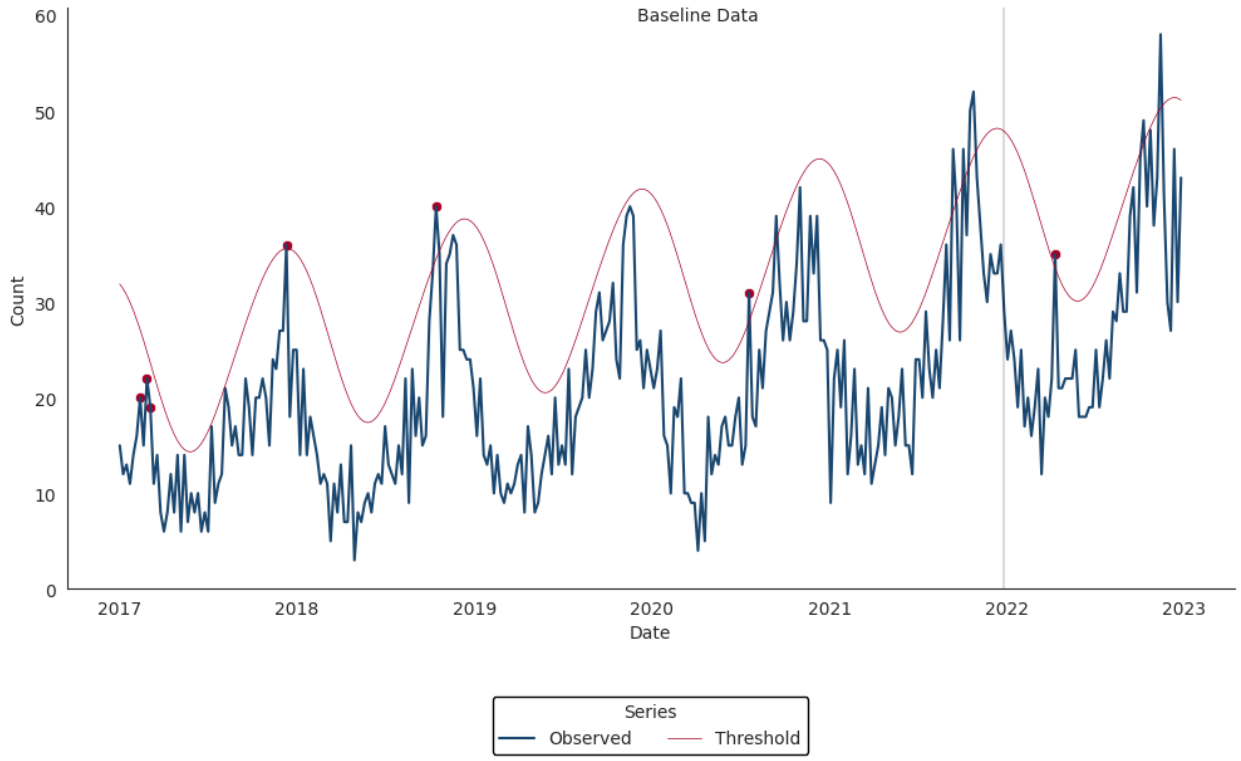
# Set axis labels and limits
>>> ax.set_xlabel('Date')
>>> ax.set_ylabel('Count')
>>> ax.set_ylim(bottom=0)
>>> ax.yaxis.set_major_formatter(
...     plt.FuncFormatter(lambda x, loc: "{:,}".format(int(x)))
... )

# Set legend
>>> ax.legend(
...     title="Series", loc='lower center', bbox_to_anchor=(0.5, -0.3),
...     ncol=2, framealpha=1, edgecolor='black'
... )

# Set date axis
>>> ax.xaxis.set_major_locator(mdates.YearLocator())
>>> ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))

# Set tick parameters
>>> ax.tick_params(axis='both', length=2.5, width=0.5)

# Set theme
>>> sns.set_style('white')
>>> sns.despine()
```




## 4.9 Trend Classification

The trend classification fits rolling binomial models to a daily time series of proportions in order to classify local trend during the baseline period as significantly increasing, significantly decreasing, or stable.

The algorithm can be performed via the `classify_trend()` function (run `help(classify_trend)` or `classify_trend?` in Jupyter Notebook or JupyterLab for more).

The test statistic and p-value are extracted from each individual model and are used in the following classification scheme:

- p-value < 0.01 and sign(test_statistic) > 0 ~ "Significant Increase"
- p-value < 0.01 and sign(test_statistic) < 0 ~ "Significant Decrease"
- p-value >= 0.01 ~ "Stable"

If there are fewer than 10 encounters/counts in the baseline period, a model is not fit and a value of `NaN` is returned for the test statistic and p-value.

```
# Detecting trends
>>> data_trend = classify_trend(df_hhs, data_count = "dataCount", all_
↪count = "allCount")
```

Let's visualize HHS Region 4 trends with the color bar at the bottom representing the trend classification of each day over time:

```
# Filter the dataset
>>> df_trend_region = (
...     data_trend.loc[data_trend["hospitaldhhsregion_display"] ==
↪"Region 4"]
...     .assign(proportion=lambda d: d["dataCount"] / d["allCount"])
... )

# Convert and sort dates
>>> df_trend_region["date"] = pd.to_datetime(df_trend_region["date"])
>>> df_trend_region = df_trend_region.sort_values("date")
```

```
# Trend tile colors (matching the R `scale_xfill_manual`)
>>> trend_colors = {
...     "Significant Increase": "#D7191C",
...     "Significant Decrease": "#2C7BB6",
...     "Stable": "#FFFFBF",
...     "Insufficient Data": "#DDDDDD",
... }

# Optional palette you defined in R (kept for parity)
>>> pal = ["#FF0000", "#1D8AFF", "#FFF70E", "grey90"]

>>> sns.set_style("white")

# Two stacked axes: main line + thin trend strip
>>> fig = plt.figure(figsize=(10, 6), dpi=150)
>>> gs = fig.add_gridspec(nrows=2, ncols=1, height_ratios=[20, 1],␣
↪hspace=0.03)

>>> ax = fig.add_subplot(gs[0, 0])
>>> ax_strip = fig.add_subplot(gs[1, 0], sharex=ax)

# Main trend line
>>> ax.plot(df_trend_region["date"], df_trend_region["proportion"], color=
↪"#1A476F", linewidth=1.4)

# Y axis as percent-like labels (0%, 2%, 4%, 6%)
>>> ax.set_ylabel("Percent", fontsize=14)
>>> ax.set_ylim(0, max(0.063, df_trend_region["proportion"].max() * 1.2))
>>> ax.yaxis.set_major_locator(plt.MultipleLocator(0.02))
>>> ax.yaxis.set_major_formatter(lambda v, p: f"{int(round(v * 100))}%")
```

(continues on next page)

(continued from previous page)

```
# Classic-like theme
>>> for spine in ["top", "right"]:
...     ax.spines[spine].set_visible(False)

>>> ax.spines["left"].set_linewidth(1.2)
>>> ax.spines["bottom"].set_linewidth(1.2)
>>> ax.tick_params(axis="both", width=1.2, length=8, labelsize=11)

# Build week-width tiles
>>> dates = df_trend_region["date"].to_list()
>>> if len(dates) > 1:
...     width = (df_trend_region["date"].diff().dropna().median())
... else:
...     width = pd.Timedelta(days=7)

>>> for _, r in df_trend_region.iterrows():
...     color = trend_colors.get(r["trend_classification"], "#DDDDDD")
...     left = r["date"] - width / 2
...     right = r["date"] + width / 2
...     ax_strip.axvspan(left, right, color=color, ec=color)

>>> ax_strip.set_ylim(0, 1)
>>> ax_strip.set_yticks([0.5])
>>> ax_strip.set_yticklabels(["Trend"], fontsize=13)
>>> ax_strip.tick_params(axis="y", length=0)

>>> for spine in ["top", "right"]:
...     ax_strip.spines[spine].set_visible(False)

>>> ax_strip.spines["left"].set_linewidth(1.2)
>>> ax_strip.spines["bottom"].set_linewidth(1.2)

# X axis formatting
>>> ax_strip.xaxis.set_major_locator(mdates.WeekdayLocator(interval=1))
>>> ax_strip.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m-%d"))
>>> plt.setp(ax_strip.get_xticklabels(), rotation=90, ha="center",␣
↪fontsize=11)
>>> ax_strip.set_xlabel("Date", fontsize=14)

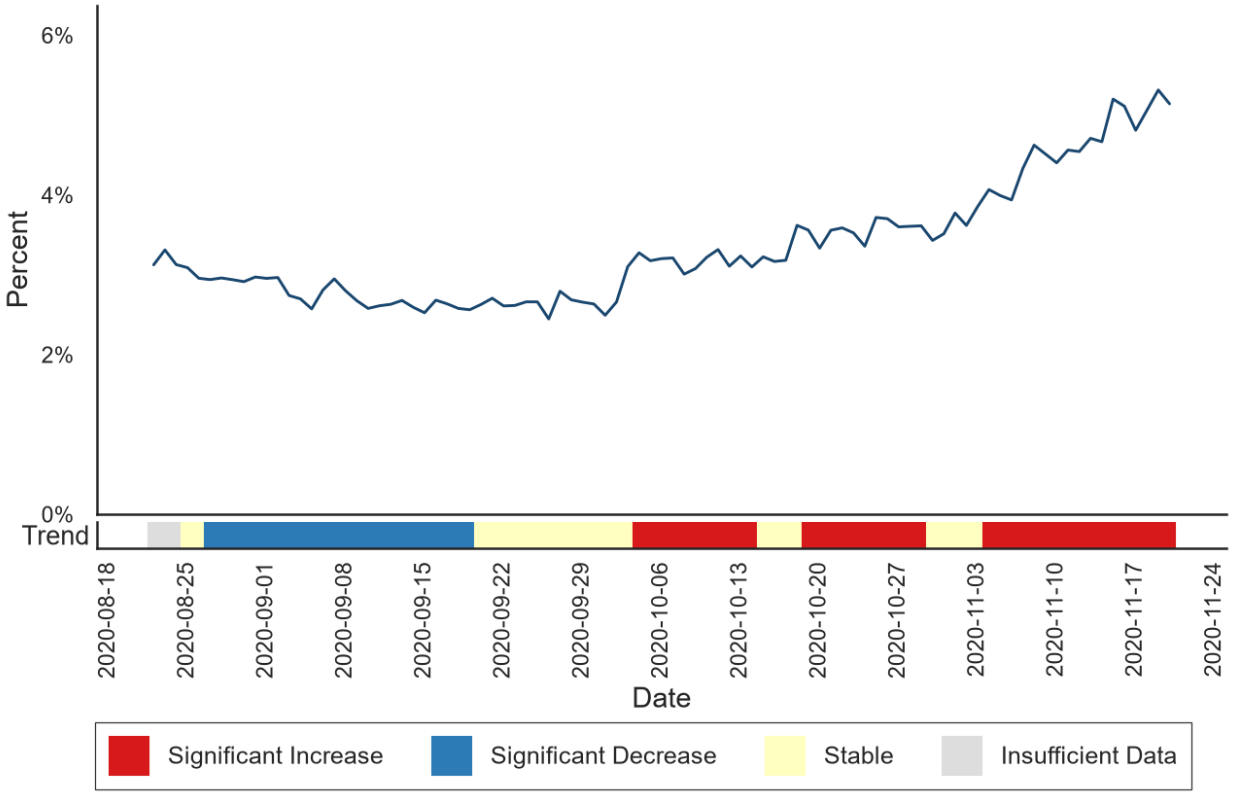
# Hide top panel x labels (show only on strip)
>>> plt.setp(ax.get_xticklabels(), visible=False)
```

(continues on next page)

(continued from previous page)

```
# Legend (bottom)
>>> legend_order = [
...     "Significant Increase",
...     "Significant Decrease",
...     "Stable",
...     "Insufficient Data",
... ]
>>> handles = [Patch(facecolor=trend_colors[k], edgecolor=trend_colors[k],
↪ label=k) for k in legend_order]
>>> leg = fig.legend(
...     handles=handles,
...     loc="lower center",
...     ncol=4,
...     bbox_to_anchor=(0.5, -0.03),
...     frameon=True,
...     fancybox=False,
...     edgecolor="black",
...     fontsize=12,
...     handlelength=1.6,
...     handleheight=2.0,
...     borderpad=0.6,
... )
>>> leg.get_frame().set_linewidth(0.6)

>>> plt.subplots_adjust(bottom=0.26)
>>> plt.show()
```


6%
4%
2%
0%
Percent
Trend
2020-08-18
2020-08-25
2020-09-01
2020-09-08
2020-09-15
2020-09-22
2020-09-29
2020-10-06
2020-10-13
2020-10-20
2020-10-27
2020-11-03
2020-11-10
2020-11-17
2020-11-24
Date
Significant Increase
Significant Decrease
Stable
Insufficient Data

Chapter 5

# Text Mining Example

## 5.1 Introduction

In this tutorial, we describe how to perform basic text mining using line-level data from NSSP-ESSENCE. This example uses line-level data details from the Chief Complaint Query Validation (CCQV) data source for the CDC Coronavirus-DD definition, limited to ED visits (Has been Emergency = "Yes").

The CCQV data source includes the chief complaint and discharge diagnosis fields, and does not include facility location, patient location, or other demographic information. This data source was created with the NSSP Community of Practice (CoP) so that users can test and create new syndrome categories using a large corpus of chief complaint and diagnosis code data that includes more variation than would be present in any one site. Some sites have opted out of contributing their data to the Query Validation data source, including Arizona, Idaho, Illinois, Marion County, Indiana, Massachusetts, North Dakota, and Ohio.

We start this tutorial by loading the `pynssp` package.

```
>>> from pynssp import create_profile, get_essence_data, load_nssp_
↪stopwords
```

Then, we load additional packages for data processing and visualization purposes:

```
>>> import re
>>> from collections import Counter
>>> from itertools import combinations
>>> from pathlib import Path
>>> import numpy as np
>>> import pandas as pd
>>> import seaborn as sns
>>> import matplotlib.pyplot as plt
```

(continues on next page)

(continued from previous page)

```
>>> import networkx as nx
```

Next, we create an NSSP user profile.

```
>>> # Creating an ESSENCE user profile
>>> myProfile = create_profile()
>>>
>>> # save profile object to file for future use
>>> # myProfile.pickle()
>>>
>>> # Load profile object
>>> # myProfile = pd.read_pickle("myProfile.pkl")
```

The above code needs to be executed only once. Upon execution, it prompts the user to provide their username and password.

Alternatively, the `myProfile` object can be saved as a `.pkl` (`.pickle`) file and loaded for future use.

## 5.2 Data Pull from NSSP-ESSENCE

We now use the `myProfile` object to authenticate to NSSP-ESSENCE and pull the data.

```
>>> url = "https://essence2.syndromicsurveillance.org/nssp_essence/api/
↪dataDetails/csv?endDate=31Dec2020&ccddCategory=cdc%20coronavirus-dd
↪%20v1&timeResolution=weekly&hasBeenE=1&percentParam=noPercent&
↪userId=2362&datasource=va_erccdd&aqtTarget=DataDetails&
↪detector=nodetectordetector&startDate=30Dec2020"
>>>
>>> api_data = get_essence_data(url, profile=myProfile)
```

We perform a standard series of text cleansing and pre-processing operations to remove meaningless patterns and common stop words, standardize free text, and return a clean version of the original dataframe that is well suited for summarizing a large corpus.

```
>>> stopwords_pattern = r"(?i)\b(?
↪:OF|OR|WITH|FOR|ON|LEFT|RIGHT|PT|UNSPECIFIED|STATUS|IN|HX|DAYS|AGO|UNKNOWN|TNOTE|201
↪b"
>>>
>>> pattern1 = r"\b(?:[A-Z][0-9]{2}[A-Z0-9]?(?:\.[0-9A-Z]{1,4})?|PG[0-9]
↪{1,3})\b"
>>> pattern2 = r"[\x00-\x1F]|<BR>|[#?!?.'+)(:=@%]"
```

(continues on next page)

(continued from previous page)

```
>>> pattern3 = r"\b(?:\d{1,2}|NA)\b|\|"
>>> pattern4 = r"[;/,\\*\-]|(?<=\w)/(?=\w)"
>>> pattern5 = r"\b[A-Za-z]\b"
>>>
>>> def uniq_words(text: str) -> str:
...     words = [w for w in str(text).split() if w]
...     return " ".join(dict.fromkeys(words))
...
>>> for col in ["ChiefComplaintParsed", "DischargeDiagnosis"]:
...     if col not in data.columns:
...         data[col] = ""
...
>>> data["chief_complaint_parsed"] = (
...     data["ChiefComplaintParsed"].fillna("").astype(str)
...     .str.replace(stopwords_pattern, " ", regex=True)
...     .str.replace(pattern1, " ", regex=True)
...     .str.replace(r"CHIEF COMPLAINT|SEE CHIEF", " ", regex=True)
...     .str.replace(pattern2, " ", regex=True)
...     .str.replace(pattern3, " ", regex=True)
...     .str.replace(pattern4, " ", regex=True)
...     .str.replace(r"\\s+", " ", regex=True)
...     .str.strip()
... )
>>> data.loc[data["chief_complaint_parsed"].str.len() <= 3, "chief_
↪complaint_parsed"] = ""
>>>
>>> data["discharge_diagnosis"] = (
...     data["DischargeDiagnosis"].fillna("").astype(str)
...     .str.replace(pattern2, " ", regex=True)
...     .str.replace(pattern3, " ", regex=True)
...     .str.replace(pattern4, " ", regex=True)
...     .str.replace(r"\\s+", " ", regex=True)
...     .str.strip()
...     .str.replace(pattern5, " ", regex=True)
...     .str.replace(r"\\s+", " ", regex=True)
...     .str.strip()
... )
>>> data.loc[data["discharge_diagnosis"].str.len() <= 2, "discharge_
↪diagnosis"] = ""
>>>
>>> data["ccdd2"] = (data["chief_complaint_parsed"] + " " + data[
↪"discharge_diagnosis"]).str.replace(r"\\s+", " ", regex=True).str.
↪strip()
```

(continues on next page)

(continued from previous page)

```
>>> data["chief_complaint_parsed"] = data["chief_complaint_parsed"].
↪map(uniq_words)
>>> data["discharge_diagnosis"] = data["discharge_diagnosis"].map(uniq_
↪words)
>>> data["ccdd2"] = data["ccdd2"].map(uniq_words)
>>> data["linenumber"] = np.arange(1, len(data) + 1)
>>>
>>> data[["chief_complaint_parsed", "discharge_diagnosis", "ccdd2",
↪"linenumber"]]
>>>
>>> # Preview the `data` dataframe
>>> # data.head()
```

## 5.3 Tokenization

After cleaning and preparing free-text fields, the next step in most natural language processing tasks is tokenization.

Tokenization is the process of partitioning sentences, paragraphs, or documents into smaller units such as individual words (unigrams) or contiguous sequences of n words (n-grams). These smaller units are referred to as tokens and provide a feature-based representation of text.

We load NSSP stopwords from the `pynssp` package to exclude a curated list of stopwords provided by the Community of Practice of the National Syndromic Surveillance Program (NSSP).

```
>>> # Load NSSP stopwords
>>> nssp_stopwords = load_nssp_stopwords()
>>>
>>> STOP_WORDS = set(nssp_stopwords["word"].astype(str).str.lower().str.
↪strip().tolist())
>>> STOP_WORDS.update({"nan", "none"})
>>>
>>> def tokenize_words(text):
...     return re.findall(r"[A-Za-z]+", str(text).lower())
...
>>> def make_ngrams(tokens, n):
...     return [" ".join(tokens[i:i+n]) for i in range(len(tokens) - n +␣
↪1)]
...
>>> rows_uni, rows_bi, rows_tri = [], [], []
>>> for _, row in data[["linenumber", "chief_complaint_parsed"]].
↪iterrows():
...     ln = row["linenumber"]
```

(continues on next page)

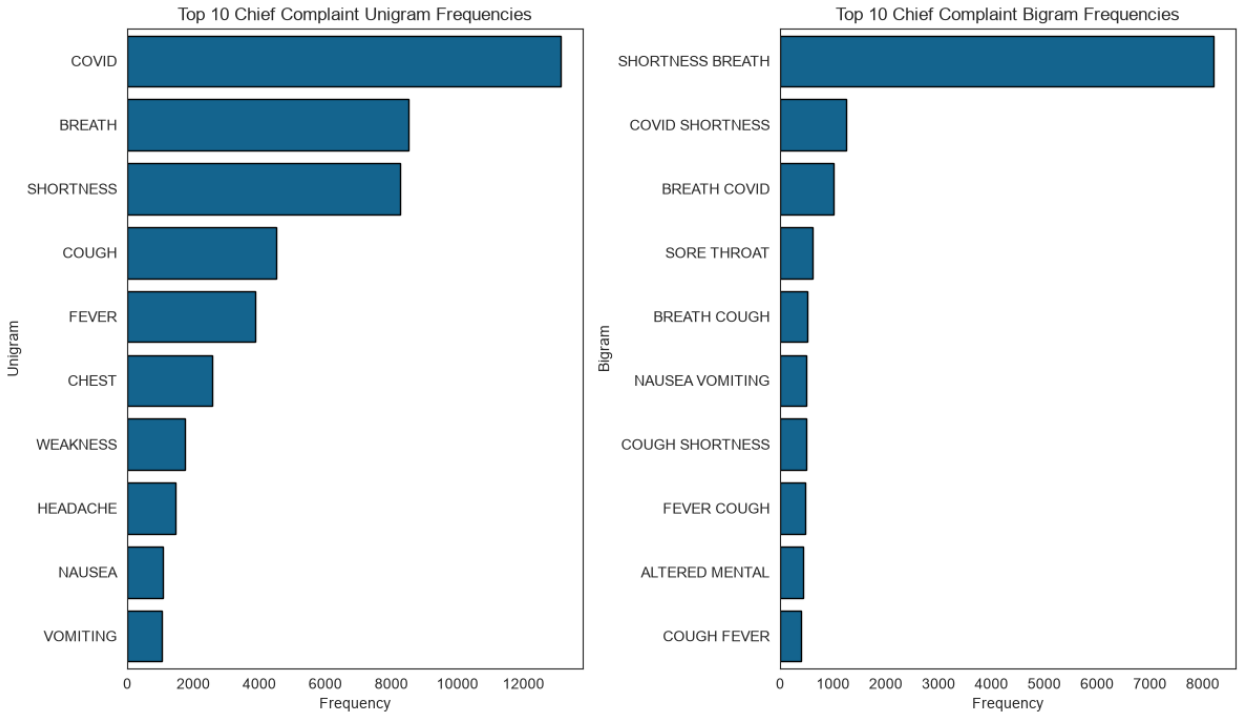
(continued from previous page)

```
...     toks = [t for t in tokenize_words(row["chief_complaint_parsed"])␣
↪if t not in STOP_WORDS]
...
...     rows_uni.extend([(ln, t) for t in toks])
...     rows_bi.extend([(ln, ng) for ng in make_ngrams(toks, 2)])
...     rows_tri.extend([(ln, ng) for ng in make_ngrams(toks, 3)])
...
>>> cc_unigrams = pd.DataFrame(rows_uni, columns=["linenumber", "word"])
>>> cc_bigrams = pd.DataFrame(rows_bi, columns=["linenumber", "bigram"])
>>> cc_trigrams = pd.DataFrame(rows_tri, columns=["linenumber", "trigram
↪"])
>>>
>>> # cc_unigrams.head(), cc_bigrams.head(), cc_trigrams.head()
```

## 5.4 n-gram Frequencies

We summarize the top 10 chief complaint unigram and bigram frequencies and visualize the results in horizontal bar charts.

```
>>> sns.set_style("white")
>>>
>>> uni_top = cc_unigrams["word"].str.upper().value_counts().head(10).
↪sort_values(ascending=False)
>>> bi_top = cc_bigrams["bigram"].str.upper().value_counts().head(10).
↪sort_values(ascending=False)
>>>
>>> fig, axes = plt.subplots(1, 2, figsize=(12, 7))
>>>
>>> sns.barplot(x=uni_top.values, y=uni_top.index, color="#006ba2",␣
↪edgecolor="black", ax=axes[0])
>>> axes[0].set_title("Top 10 Chief Complaint Unigram Frequencies")
>>> axes[0].set_xlabel("Frequency")
>>> axes[0].set_ylabel("Unigram")
>>>
>>> sns.barplot(x=bi_top.values, y=bi_top.index, color="#006ba2",␣
↪edgecolor="black", ax=axes[1])
>>> axes[1].set_title("Top 10 Chief Complaint Bigram Frequencies")
>>> axes[1].set_xlabel("Frequency")
>>> axes[1].set_ylabel("Bigram")
>>>
>>> plt.tight_layout()
>>> plt.show()
```




## 5.5 Chief Complaint Term Co-occurrence and Correlation

### 5.5.1 Chief Complaint Term Co-occurrence Network Graph

```
>>> tokens_per_line = (
...     cc_unigrams.groupby("linenumber")["word"]
...     .apply(lambda s: sorted(set(s.tolist())))
... )
>>>
>>> pair_counter = Counter()
>>> for toks in tokens_per_line:
...     for a, b in combinations(toks, 2):
...         pair_counter[(a.upper(), b.upper())] += 1
...
>>> co_df = (
...     pd.DataFrame([(a, b, n) for (a, b), n in pair_counter.items()],␣
↪columns=["item1", "item2", "n"])
...     .sort_values("n", ascending=False)
...     .head(200)
... )
>>>
>>> if co_df.empty:
...     print("No co-occurrence edges available after filtering.")
```

(continues on next page)

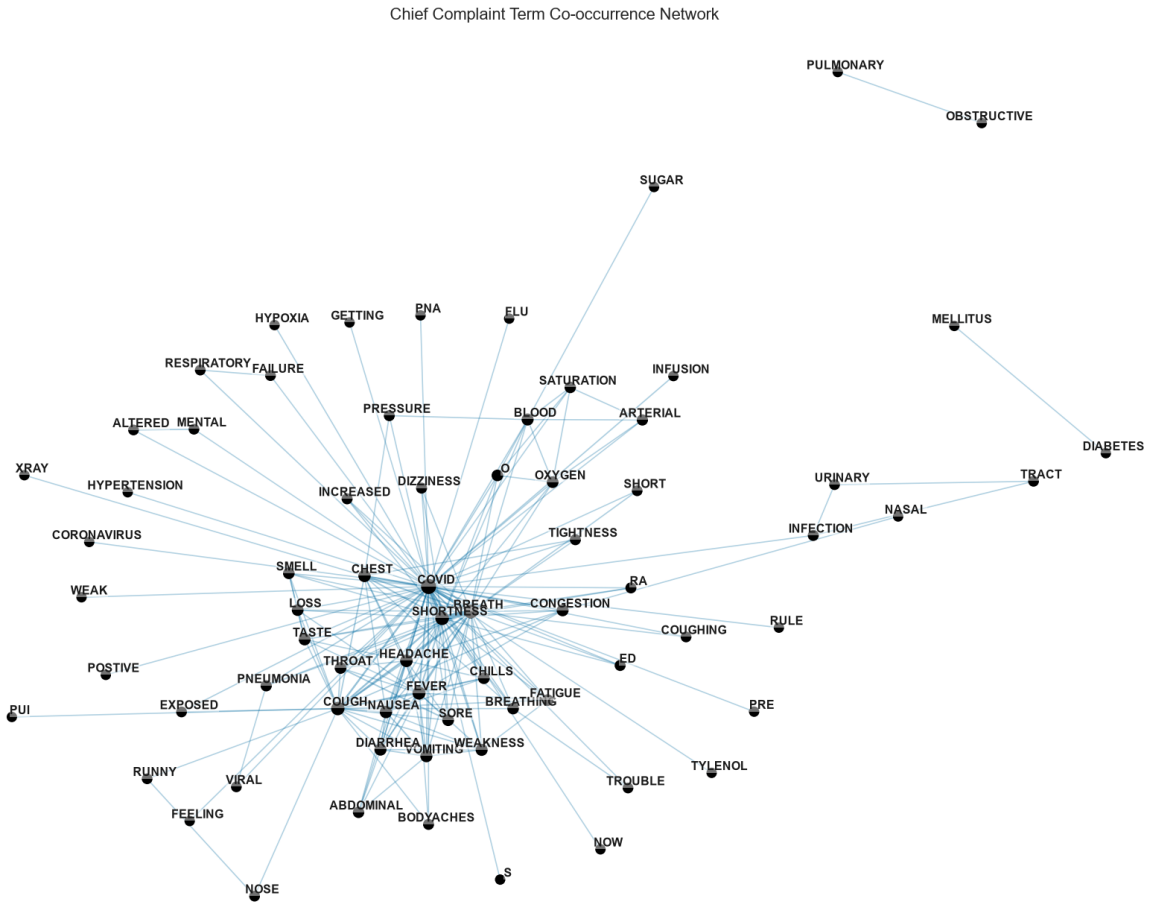
(continued from previous page)

```
... else:
...     G = nx.from_pandas_edgelist(co_df, source="item1", target="item2",
↪ edge_attr="n")
...     pos = nx.spring_layout(G, seed=42, k=0.42)
...     edge_weights = np.array([d.get("n", 1) for _, _, d in G.
↪edges(data=True)], dtype=float)
...     edge_alphas = (edge_weights - edge_weights.min()) / (edge_weights.
↪max() - edge_weights.min() + 1e-9)
...     degrees = dict(G.degree())
...     node_sizes = np.array([50 + 10 * np.sqrt(degrees[n]) for n in G.
↪nodes()], dtype=float)
...
...     plt.close("all")
...     fig, ax = plt.subplots(figsize=(13, 10), dpi=130)
...
...     for (u, v, d), a in zip(G.edges(data=True), edge_alphas):
...         nx.draw_networkx_edges(
...             G, pos, edgelist=[(u, v)],
...             alpha=0.30 + 0.65 * a,
...             width=0.9 + 2.6 * a,
...             edge_color="#006BA2",
...             ax=ax
...         )
...
...     nx.draw_networkx_nodes(G, pos, node_size=node_sizes, node_color=
↪"black", linewidths=0, ax=ax)
...
...     label_pos = {n: (xy[0] + 0.012, xy[1] + 0.012) for n, xy in pos.
↪items()}
...     labels = {n: n for n in G.nodes()}
...     nx.draw_networkx_labels(
...         G,
...         label_pos,
...         font_size=9,
...         font_weight="semibold",
...         font_color="#1a1a1a",
...         labels=labels,
...         bbox=dict(facecolor="white", edgecolor="none", alpha=0.45,␣
↪pad=0.12),
...         ax=ax,
...     )
...
...     ax.set_axis_off()
```

(continues on next page)

(continued from previous page)

```
...     ax.set_title("Chief Complaint Term Co-occurrence Network")
...     fig.tight_layout()
...     plt.show()
```




## 5.6 Pearson Pairwise Correlation Chief Complaint Network Graph

While co-occurrence measures how frequently two terms appear together in the same chief complaint, the Phi coefficient measures how strongly two terms are associated. The Phi coefficient between two words, $w_1$ and $w_2$, is defined as:

$$\phi = \frac{n_{11}n_{00} - n_{10}n_{01}}{\sqrt{n_{1\bullet}n_{0\bullet}n_{\bullet 0}n_{\bullet 1}}}.$$

In this context:

- $n_{11}$ is the number of chief complaints containing both $w_1$ and $w_2$
- $n_{00}$ is the number of chief complaints **not** containing both $w_1$ and $w_2$

- $n_{01}$ is the number of chief complaints containing $w_2$ and **not** $w_1$
- $n_{10}$ is the number of chief complaints containing $w_1$ and **not** $w_2$

| | Contains $w_2$ | Does not contain $w_2$ | Total |
|---|---|---|---|
| Contains $w_1$ | $n_{11}$ | $n_{10}$ | $n_{1\bullet}$ |
| Does not contain $w_1$ | $n_{01}$ | $n_{00}$ | $n_{0\bullet}$ |
| Total | $n_{\bullet 1}$ | $n_{\bullet 0}$ | $n$ |

```
>>> word_counts = cc_unigrams["word"].value_counts()
>>> eligible_words = set(word_counts[word_counts >= 20].index)
>>>
>>> line_sets = (
...     cc_unigrams[cc_unigrams["word"].isin(eligible_words)]
...     .groupby("linenumber")["word"]
...     .apply(lambda s: set(s.tolist()))
... )
>>>
>>> N = len(line_sets)
>>> corr_rows = []
>>>
>>> eligible = sorted(eligible_words)
>>> for i, w1 in enumerate(eligible):
...     w1_lines = line_sets[line_sets.apply(lambda s: w1 in s)].index
...     set1 = set(w1_lines)
...     for w2 in eligible[i + 1:]:
...         w2_lines = line_sets[line_sets.apply(lambda s: w2 in s)].index
...         set2 = set(w2_lines)
...
...         n11 = len(set1 & set2)
...         n10 = len(set1 - set2)
...         n01 = len(set2 - set1)
...         n00 = N - n11 - n10 - n01
...
...         denom = np.sqrt((n11 + n10) * (n01 + n00) * (n11 + n01) *␣
↪(n10 + n00))
...         if denom == 0:
...             continue
...
...         phi = (n11 * n00 - n10 * n01) / denom
...         if phi > 0.5:
...             corr_rows.append((w1.upper(), w2.upper(), phi))
```

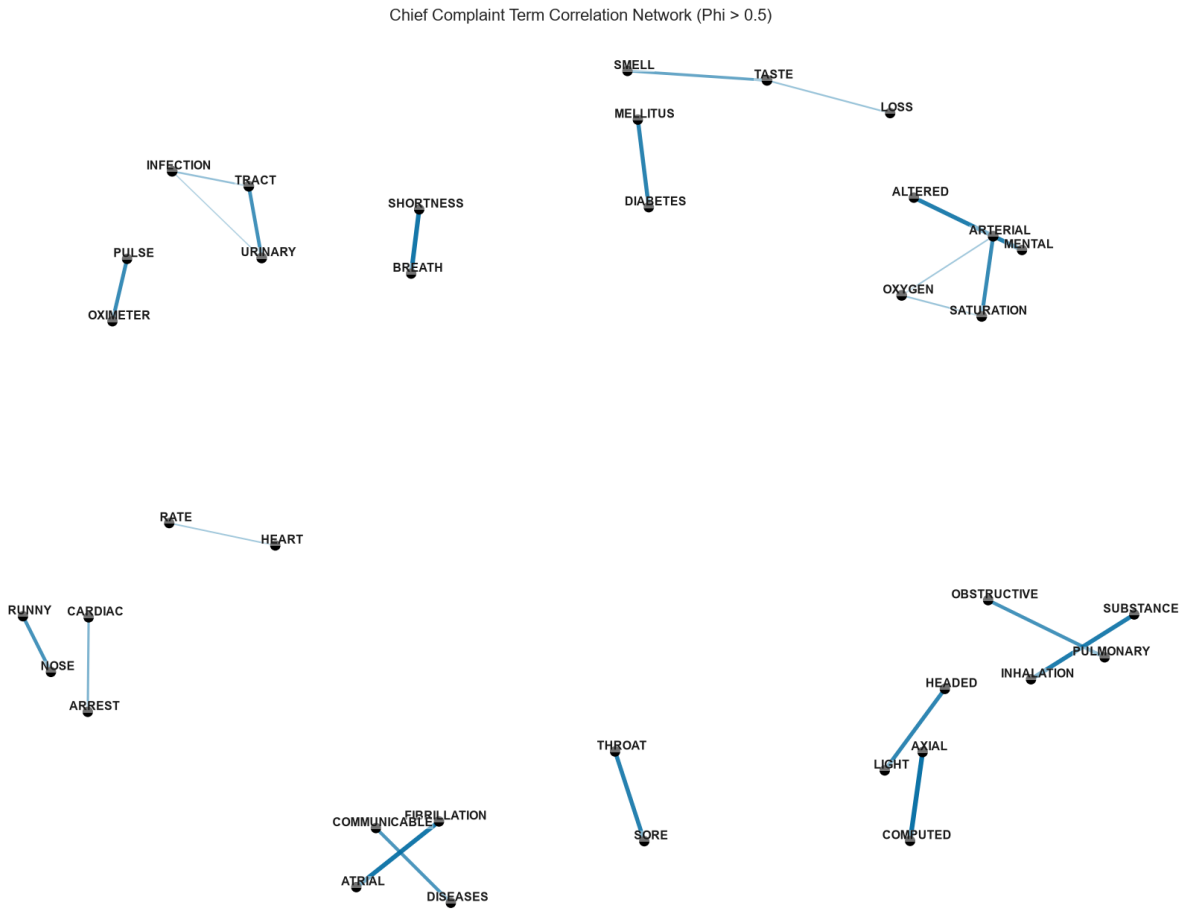
Let's visualize the network graph:

```
>>> corr_df = pd.DataFrame(corr_rows, columns=["item1", "item2",
↪"correlation"])
>>>
>>> if corr_df.empty:
...     print("No correlation edges found above the threshold (phi > 0.5).
↪")
... else:
...     G_corr = nx.from_pandas_edgelist(
...         corr_df, source="item1", target="item2", edge_attr=
↪"correlation"
...     )
...     pos = nx.spring_layout(G_corr, seed=42, k=0.42)
...
...     edge_vals = np.array(
...         [d.get("correlation", 0.0) for _, _, d in G_corr.
↪edges(data=True)],
...         dtype=float
...     )
...     edge_alphas = (edge_vals - edge_vals.min()) / (edge_vals.max() -␣
↪edge_vals.min() + 1e-9)
...
...     degrees = dict(G_corr.degree())
...     node_sizes = np.array([50 + 10 * np.sqrt(degrees[n]) for n in G_
↪corr.nodes()], dtype=float)
...
...     plt.close("all")
...     fig, ax = plt.subplots(figsize=(13, 10), dpi=130)
...
...     for (u, v, d), a in zip(G_corr.edges(data=True), edge_alphas):
...         nx.draw_networkx_edges(
...             G_corr, pos, edgelist=[(u, v)],
...             alpha=0.30 + 0.65 * a,
...             width=0.9 + 2.6 * a,
...             edge_color="#006BA2",
...             ax=ax
...         )
...
...     nx.draw_networkx_nodes(
...         G_corr, pos, node_size=node_sizes, node_color="black",␣
↪linewidths=0, ax=ax
...     )
...
```

(continues on next page)

(continued from previous page)

```
...     label_pos = {n: (xy[0] + 0.012, xy[1] + 0.012) for n, xy in pos.
↪items()}
...     labels = {n: n for n in G_corr.nodes()}
...     nx.draw_networkx_labels(
...         G_corr,
...         label_pos,
...         font_size=9,
...         font_weight="semibold",
...         font_color="#1a1a1a",
...         labels=labels,
...         bbox=dict(facecolor="white", edgecolor="none", alpha=0.45,␣
↪pad=0.12),
...         ax=ax,
...     )
...
...     ax.set_axis_off()
...     ax.set_title("Chief Complaint Term Correlation Network (Phi > 0.5)
↪")
...     fig.tight_layout()
...     plt.show()
```


Chief Complaint Term Correlation Network (Phi > 0.5)
SMELL
TASTE
LOSS
MELLITUS
DIABETES
INFECTION
TRACT
URINARY
PULSE
OXIMETER
SHORTNESS
BREATH
ALTERED
ARTERIAL
MENTAL
OXYGEN
SATURATION
RATE
HEART
RUNNY
CARDIAC
NOSE
ARREST
OBSTRUCTIVE
SUBSTANCE
PULMONARY
INHALATION
HEADED
AXIAL
LIGHT
COMPUTED
THROAT
SORE
COMMUNICABLE
FIBRILLATION
ATRIAL
DISEASES

Chapter 6

# pynssp

## 6.1 pynssp package

### 6.1.1 Subpackages

#### pynssp.core package

#### Submodules

#### pynssp.core.constants module

#### pynssp.core.container module

**class** pynssp.core.container.**APIGraph**(*path*, *response*)

Bases: object

A class to store an API graph

**Parameters**

- **path** – a string representing the location of a graph
- **response** – an object of class response

**Variables**

- **path** – the location of a graph file
- **response** – a response object

**plot**()

A method to plot an APIGraph object

**show**()

A method to display an APIGraph object

class pynssp.core.container.**NSSPContainer**(*value*)

Bases: object

A NSSPContainer Class to store a value or an object

An object of class NSSPContainer stores a value or an object The NSSPContainer class encapsulates a value or an object

**Parameters**
**value** – value to store

**Variables**
**value** – Stored value

**Examples**

```
>>> from pynssp import NSSPContainer
>>>
>>> cont = NSSPContainer("abcdef")
```

## pynssp.core.credentials module

class pynssp.core.credentials.**Credentials**(*username=None*, *password=None*)

Bases: Auth

A Token Class Representing a Credentials object

A Credentials object has a username, a password and a key. A Credentials object gets API data via an API URL.

**Parameters**

- **username** – a string for username (Default username = None)
- **password** – a string for password (Default password = None)

**Examples**

```
>>> from pynssp import Credentials
>>>
>>> myProfile = Credentials("user", "pass")
```

**get_api_response**(*url*)

Get API response

**Parameters**
**url** – a string of API URL

**Returns**
an object of class response

### pynssp.core.token module

**class** pynssp.core.token.**Token**(*token*, *access_token='Bearer'*)

Bases: Auth

A Token Class Representing a Token object

A Token object has a token string and a key. A Token object can get API data via an API URL.

**Parameters**

- **token** – a token string
- **access_token** – type of HTTP authentication. Should be "Bearer" or "Basic". (Default value = "Bearer")

**Variables**

**access_token** – HTTP authentication type.

**Examples**

```
>>> from pynssp import Token
>>>
>>> myTokenProfile = Token("abc123")
```

**get_api_response**(*url*)

Get API response

**Parameters**

**url** – a string of API URL

**Returns**

an object of class response

## Module contents

## pynssp.detectors package

## Submodules

### pynssp.detectors.ewma module

pynssp.detectors.ewma.**alert_ewma**(*df*, *t='date'*, *y='count'*, *B=28*, *g=2*, *w1=0.4*, *w2=0.9*)

Exponentially Weighted Moving Average (EWMA)

The EWMA compares a weighted average of the most recent visit counts to a baseline expectation. For the weighted average to be tested, an exponential weighting gives the most influence to the most recent observations. This algorithm is appropriate for daily counts that

do not have the characteristic features modeled in the regression algorithm. It is more applicable for Emergency Department data from certain hospital groups and for time series with small counts (daily average below 10) because of the limited case definition or chosen geographic region. An alert (red value) is signaled if the statistical test (student"s t-test) applied to the test statistic yields a p-value less than 0.01. If the p-value is greater than or equal to 0.01 and strictly less than 0.05, a warning (yellow value) is signaled. Blue values are returned if an alert or warning does not occur. Grey values represent instances where anomaly detection did not apply (i.e., observations for which baseline data were unavailable).

**Parameters**

- **df** – A pandas data frame containing time series data
- **t** – Name of the column of type Date containing the dates (Default value = "date")
- **y** – Name of the column of type Numeric containing counts or percentages (Default value = "count")
- **B** – Baseline parameter. The baseline length is the number of days used to calculate rolling averages, standard deviations, and exponentially weighted moving averages. Defaults to 28 days to match ESSENCE implementation.
- **g** – Guardband parameter. The guardband length is the number of days separating the baseline from the current test date. Defaults to 2 days to match ESSENCE implementation.
- **w1** – Smoothing coefficient for sensitivity to gradual events. Must be between 0 and 1 and is recommended to be between 0.3 and 0.5 to account for gradual effects. Defaults to 0.4 to match ESSENCE implementation.
- **w2** – Smoothed coefficient for sensitivity to sudden events. Must be between 0 and 1 and is recommended to be above 0.7 to account for sudden events. Defaults to 0.9 to match ESSENCE implementation and approximate the C2 algorithm.

**Returns**

Original pandas data frame with detection results.

**Examples**

```
>>> from pynssp import alert_ewma
>>> import pandas as pd
>>> import numpy as np
>>>
>>> df = pd.DataFrame({
...     "date": pd.date_range("2020-01-01", "2020-12-31"),
...     "count": np.random.randint(0, 101, size=366)
```

(continues on next page)

(continued from previous page)

```
... })
>>>
>>> df_ewma = alert_ewma(df)
>>> df_ewma.head()
```

pynssp.detectors.ewma.**ewma_loop**(*df*, *t*, *y*, *B*, *g*, *w1*, *w2*)

Loop for EWMA

Loop for EWMA and adjustment of outlying smoothed values

**Parameters**

- **df** – A pandas data frame
- **t** – Name of the column of type Date containing the dates
- **y** – Name of the column containing the response variable data
- **mu** – Numeric vector of baseline averages
- **B** – Baseline parameter. The baseline length is the number of days used to calculate rolling averages, standard deviations, and exponentially weighted moving averages. Defaults to 28 days to match NSSP-ESSENCE implementation.
- **g** – Guardband parameter. The guardband length is the number of days separating the baseline from the current test date. Defaults to 2 days to match NSSP-ESSENCE implementation.
- **w1** – Smoothing coefficient for sensitivity to gradual events. Must be between 0 and 1 and is recommended to be between 0.3 and 0.5 to account for gradual effects. Defaults to 0.4 to match NSSP-ESSENCE implementation.
- **w2** – Smoothed coefficient for sensitivity to sudden events. Must be between 0 and 1 and is recommended to be above 0.7 to account for sudden events. Defaults to 0.9 to match NSSP-ESSENCE implementation and approximate the C2 algorithm.

**Returns**

A pandas data frame with p-values and test statistics

### pynssp.detectors.nbinom module

pynssp.detectors.nbinom.**alert_nbinom**(*df*, *baseline_end*, *t='date'*, *y='count'*, *include_time=True*)

Negative binomial detection algorithm for weekly counts

The negative binomial regression algorithm fits a negative binomial regression model with a time term and order 1 Fourier terms to a baseline period that spans 2 or more years. Inclusion of Fourier terms in the model is intended to account for seasonality common in multi-year weekly time series of counts. Order 1 sine and cosine terms are included to account for annual seasonality that is common to syndromes and diseases such as influenza, RSV, and norovirus. Each baseline model is used to make weekly forecasts for all weeks following the baseline period. One-sided upper 95% prediction interval bounds are computed for each week in the prediction period. Alarms are signaled for any week during for which weekly counts fall above the upper bound of the prediction interval.

**Parameters**

- **df** – A pandas data frame containing time series data
- **t** – Name of the column of type Date containing the dates (Default value = "date")
- **y** – Name of the column of type Numeric containing counts or percentages (Default value = "count")
- **baseline_end** – date of the end of the baseline/training period (in date or string class)
- **include_time** – Indicate whether or not to include time term in regression model (default is True)

**Returns**

Original pandas dataframe with model estimates, upper prediction interval bounds, a binary alarm indicator field, and a binary indicator field of whether or not a time term was included.

**Examples**

```
>>> from pnssp import alert_nbinom
>>> import pandas as pd
>>> import numpy as np
>>>
>>> df = pd.DataFrame({
...     "date": pd.date_range(start="2014-01-05", end=
↪"2022-02-05", freq="W"),
...     "count": np.random.poisson(lam=25, size=(len(pd.
↪date_range(start="2014-01-05", end="2022-02-05", freq="W
↪")),))
... })
>>>
>>> df_nbinom = alert_nbinom(df, baseline_end = "2020-03-
↪01")
>>> df_nbinom.head()
```

pynssp.detectors.nbinom.**nb_model**(*df*, *t*, *y*, *baseline_end*, *include_time*)

Negative binomial regression model for weekly counts

Negative binomial model helper function for monitoring weekly count time series with seasonality

**Parameters**

- **df** – A pandas data frame
- **t** – Name of the column of type Date containing the dates
- **y** – Name of the column of type Numeric containing counts
- **baseline_end** – Object of type Date defining the end of the baseline/training period
- **include_time** – Logical indicating whether or not to include time term in regression model

**Returns**

A pandas data frame.

### pynssp.detectors.regression module

pynssp.detectors.regression.**adaptive_regression**(*df*, *t*, *y*, *B*, *g*)

Adaptive Regression

Adaptive Regression helper function for Adaptive Multiple Regression

**Parameters**

- **df** – A pandas data frame containing time series data
- **t** – Name of the column of type Date containing the dates
- **y** – Name of the column containing the response variable data
- **B** – Baseline parameter. The baseline length is the number of days to which each liner model is fit.
- **g** – Guardband parameter. The guardband length is the number of days separating the baseline from the current date in consideration for alerting.

**Returns**

A pandas data frame with p-values and test statistics

pynssp.detectors.regression.**alert_regression**(*df*, *t='date'*, *y='count'*, *B=28*, *g=2*)

Adaptive Multiple Regression

The adaptive multiple regression algorithm fits a linear model to a baseline of counts or percentages of length B, and forecasts a predicted value g + 1 days later (guard-band). This value is compared to the current observed value and divided by the standard error of prediction in

the test-statistic. The model includes terms to account for linear trends and day-of-week effects. Note that this implementation does NOT account for federal holidays as in the Regression 1.2 algorithm in ESSENCE. An alert (red value) is signaled if the statistical test (student"s t-test) applied to the test statistic yields a p-value less than 0.01. If the p-value is greater than or equal to 0.01 and strictly less than 0.05, a warning (yellow value) is signaled. Blue values are returned if an alert or warning does not occur. Grey values represent instances where anomaly detection did not apply (i.e., observations for which baseline data were unavailable).

**Parameters**

- **df** – A pandas data frame containing time series data
- **t** – The name of the column in df that contains the dates or times of observations. Defaults to "date".
- **y** – The name of the column in df that contains the values of the time series. Defaults to "count".
- **B** – The length of the baseline period (in days). Must be a multiple of 7 and at least 7. Defaults to 28.
- **g** – The length of the guard band (in days). Must be non-negative. Defaults to 2.

**Returns**

Original pandas data frame with detection results.

**Examples**

```
>>> from pynssp import alert_regression
>>> import pandas as pd
>>> import numpy as np
>>>
>>> df = pd.DataFrame({
...     "date": pd.date_range("2020-01-01", "2020-12-31"),
...     "count": np.random.randint(0, 101, size=366)
... })
>>>
>>> df_regression = alert_regression(df)
>>> df_regression.head()
```

## pynssp.detectors.serfling module

pynssp.detectors.serfling.**alert_serfling**(*df*, *baseline_end*, *t='date'*, *y='count'*)

Original Serfling method for weekly time series

The original Serfling algorithm fits a linear regression model with a time term and order 1 Fourier terms to a baseline period that ideally spans 5 or more years. Inclusion of Fourier

terms in the model is intended to account for seasonality common in multi-year weekly time series. Order 1 sine and cosine terms are included to account for annual seasonality that is common to syndromes and diseases such as influenza, RSV, and norovirus. Each baseline model is used to make weekly forecasts for all weeks following the baseline period. One-sided upper 95% prediction interval bounds are computed for each week in the prediction period. Alarms are signaled for any week during for which weekly observations fall above the upper bound of the prediction interval. This implementation follows the approach of the original Serfling method in which weeks between October of the starting year of a season and May of the ending year of a season are considered to be in the epidemic period. Weeks in the epidemic period are removed from the baseline prior to fitting the regression model.

**Parameters**

- **df** – A pandas data frame containing time series data
- **t** – Name of the column of type Date containing the dates (Default value = "date")
- **y** – Name of the column of type Numeric containing counts or percentages (Default value = "count")
- **baseline_end** – date of the end of the baseline/training period (in date or string class)

**Returns**

Original pandas dataframe with model estimates, upper prediction interval bounds, a binary alarm indicator field, and a binary indicator

**Examples**

```
>>> from pynssp import alert_serfling
>>> import pandas as pd
>>> import numpy as np
>>>
>>> df = pd.DataFrame({
...     "date": pd.date_range(start="2014-01-05", end=
↪"2022-02-05", freq="W"),
...     "count": np.random.poisson(lam=25, size=(len(pd.
↪date_range(start="2014-01-05", end="2022-02-05", freq="W
↪")),))
... })
>>>
>>> df_serfling  = alert_serfling(df, baseline_end =
↪"2020-03-01")
>>> df_serfling.head()
```

pynssp.detectors.serfling.**serfling_model**(*df*, *t*, *y*, *baseline_end*)

Original Serfling method for weekly time series

Serfling model helper function for monitoring weekly time series with seasonality

**Parameters**

- **df** – A pandas data frame
- **t** – Name of the column of type Date containing the dates
- **y** – Name of the column of type Numeric containing counts
- **baseline_end** – date of the end of the baseline/training period (date or string class)

**Returns**

A pandas data frame.

### pynssp.detectors.switch module

pynssp.detectors.switch.**alert_switch**(*df*, *t='date'*, *y='count'*, *B=28*, *g=2*, *w1=0.4*, *w2=0.9*)

Regression/EWMA Switch

The NSSP-ESSENCE Regression/EWMA Switch algorithm generalized the Regression and EWMA algorithms by applying the most appropriate algorithm for the data in the baseline. First, multiple adaptive regression is applied where the adjusted R squared value of the model is examined to see if it meets a threshold of 0.60. If this threshold is not met, then the model is considered to not explain the data well. In this case, the algorithm switches to the EWMA algorithm, which is more appropriate for sparser time series that are common with county level trends. The smoothing coefficient for the EWMA algorithm is fixed to 0.4.

**Parameters**

- **df** – A dataframe containing the time series data.
- **t** – The name of the column in *df* containing the time information. Defaults to "date".
- **y** – The name of the column in *df* containing the values to be analyzed. Defaults to "count".
- **B** – The length of the baseline period in days, must be a multiple of 7 and greater than or equal to 7. Defaults to 28.
- **g** – The length of the guardband period in days. Must be non-negative. Defaults to 2.
- **w1** – Smoothing coefficient for sensitivity to gradual events. Must be between 0 and 1 and is recommended to be between 0.3 and 0.5 to account for gradual effects. Defaults to 0.4 to match NSSP-ESSENCE implementation.

- **w2** – Smoothed coefficient for sensitivity to sudden events. Must be between 0 and 1 and is recommended to be above 0.7 to account for sudden events. Defaults to 0.9 to match NSSP-ESSENCE implementation and approximate the C2 algorithm.

**Returns**

A dataframe containing the results of the analysis.

**Examples**

```
>>> from pynssp import alert_switch
>>> import pandas as pd
>>> import numpy as np
>>>
>>> df = pd.DataFrame({
...    "date": pd.date_range("2020-01-01", "2020-12-31"),
...    "count": np.random.randint(0, 101, size=366)
... })
>>>
>>> df_switch = alert_switch(df)
>>> df_switch.head()
```

## pynssp.detectors.trend module

pynssp.detectors.trend.**classify_trend**(*df*, *t='date'*, *data_count='dataCount'*, *all_count='allCount'*, *B=12*)

Trend Classification for Proportions/Percentages

The algorithm fits rolling binomial models to a daily time series of percentages or proportions in order to classify the overall trend during the baseline period as significantly increasing, significantly decreasing, or stable.

**Parameters**

- **df** – A pandas data frame
- **t** – Name of the column of type Date containing the dates (Default value = "date")
- **data_count** – Name of the column with counts for positive encounters (Default value = "dataCount")
- **all_count** – Name of the column with total counts of encounters (Default value = "allCount")
- **B** – Baseline parameter. The baseline length is the number of days to which each binomial model is fit (Default value = 12)

**Returns**
A pandas data frame

**Examples**

```
>>> from pynssp import classify_trend
>>> import pandas as pd
>>> import numpy as np
>>>
>>> df = pd.DataFrame({
...     "date": pd.date_range("2020-01-01", "2020-12-31"),
...     "dataCount": np.random.randint(0, 101, size=366),
...     "allCount": np.random.randint(101, 500, size=366)
... })
>>>
>>> df_trend = classify_trend(df)
>>> df_trend.head()
```

pynssp.detectors.trend.**get_trends**(*df*, *t*, *data_count*, *all_count*, *B*)

Trend Classification Helper

Fits rolling binomial models to a daily time series of percentages or proportions in order to classify the overall trend during the baseline period as significantly increasing, significantly decreasing, or stable.

**Parameters**

- **df** – A pandas data frame
- **t** – Name of the column of type Date containing the dates
- **data_count** – Name of the column with counts for positive encounters
- **all_count** – Name of the column with total counts of encounters
- **B** – Baseline parameter. The baseline length is the number of days to which each binomial model is fit

**Returns**
A pandas data frame

### Module contents

## 6.1.2 Submodules

## 6.1.3 pynssp.data module

pynssp.data.**get_scenario1**()

Return a subset of the simulated time series data ('scenario #1').

**Examples**

```
>>> from pynssp import get_scenario1
>>>
>>> scenario1_ts = get_scenario1()
>>> scenario1_ts.info()
```

pynssp.data.**get_scenario2**()

Return a subset of the simulated time series data ('scenario #2').

**Examples**

```
>>> from pynssp import get_scenario2
>>>
>>> scenario2_ts = get_scenario2()
>>> scenario2_ts.info()
```

pynssp.data.**load_nssp_stopwords**()

Return a dataframe of NSSP-curated stopwords.

**Examples**

```
>>> from pynssp import load_nssp_stopwords
>>>
>>> stopwords = load_nssp_stopwords()
>>> stopwords.info()
## #   Column      Non-Null Count  Dtype
## ---  ------      --------------  -----
## 1   word        835 non-null    object
## 2   type        835 non-null    object
## dtypes: int64(1), object(2)
## memory usage: 13.2+ KB
```

pynssp.data.**load_simulated_ts**()

Return a dataframe of simulated time series.

**Examples**

```
>>> from pynssp.data import load_simulated_ts
>>>
>>> simulated_ts = load_simulated_ts()
>>> simulated_ts.info()
## #   Column  Non-Null Count  Dtype
## ---  ------  --------------  -----
## 0   date    626 non-null    object
## 1   week    626 non-null    int64
```

(continues on next page)

(continued from previous page)

```
## 2   year    626 non-null    int64
## 3   cases   626 non-null    int64
## 4   id      626 non-null    object
## dtypes: int64(3), object(2)
## memory usage: 24.6+ KB
```

### 6.1.4 pynssp.utils module

pynssp.utils.**change_dates**(*url*, *start_date=None*, *end_date=None*)

Changes the start and end dates in a given URL to new dates, if provided.

**Parameters**

- **url** – str): The URL containing the start and end dates to be changed.
- **start_date** – str): A new start date to replace the existing start date in the URL. (Default value = None)
- **end_date** – str): A new end date to replace the existing end date in the URL. (Default value = None)

**Returns**

The modified URL with the new start and end dates.

**Examples**

```
>>> from pynssp import change_dates
>>>
>>> url = "https://example.com/data?startDate=01Jan2022&
↪endDate=31Dec2022"
>>> change_dates(url, start_date="01Jan2021", end_date=
↪"31Dec2021")
```

pynssp.utils.**create_apikey_profile**(*api_key=None*, *key_name='API-KEY'*)

Create a new apikey profile with the given api-key and key name.

**Parameters**

- **api_key** – A string representing the token. If not provided, the user will be prompted to enter it securely.
- **key_name** – A string representing the API key name. Defaults to "API-KEY".

**Returns**

A new Apikey object with the given API key and API key name.

**Examples**

```
>>> from pynssp import create_apikey_profile
>>>
>>> myApikeyProfile = create_apikey_profile()
```

pynssp.utils.**create_profile**(*username=None*, *password=None*)

Create a new user profile with the given username and password.

**Parameters**

- **username** – A string representing the username. If not provided, the user will be prompted to enter it.
- **password** – A string representing the user's password. If not provided, the user will be prompted to enter it securely.

**Returns**

A new Credentials object with the given username and password.

**Examples**

```
>>> from pynssp import create_profile
>>>
>>> myProfile = create_profile()
```

pynssp.utils.**create_token_profile**(*token=None*, *access_token='Bearer'*)

Create a new token profile with the given token and authentication type.

**Parameters**

- **token** – A string representing the token. If not provided, the user will be prompted to enter it securely.
- **auth_type** – A string representing the authentication type. Defaults to "Bearer".

**Returns**

A new Token object with the given token and authentication type.

**Examples**

```
>>> from pynssp import create_token_profile
>>>
>>> myTokenProfile = create_token_profile()
```

pynssp.utils.**get_api_data**(*url*, *fromCSV=False*, *profile=None*, *encoding='utf-8'*, ***kwargs*)

Retrieve data from an API using the provided profile.

**Parameters**

- **url** – A string representing the URL of the API endpoint.
- **fromCSV** – A boolean indicating whether the data should be retrieved from a CSV file. Defaults to False.
- **profile** – An profile object of class *pynssp.core.credentials.Credentials* or *pynssp.core.token.Token*.
- **kwargs** – Additional keyword arguments to be passed to the profile's get_api_data method.

**Returns**

The data retrieved from the API.

**Examples**

```
>>> from pynssp import *
>>>
>>> myProfile = create_profile()
>>> url = "http://httpbin.org/json"
>>> api_data = get_api_data(url, profile=myProfile)
```

pynssp.utils.**get_api_graph**(*url*, *file_ext='.png'*, *profile=None*)

Retrieve a graph from an API using the provided profile.

**Parameters**

- **url** – A string representing the URL of the API endpoint.
- **file_ext** – A string representing the file extension of the graph. Defaults to ".png".
- **profile** – An profile object of class *pynssp.core.credentials.Credentials* or *pynssp.core.token.Token*.

**Returns**

The graph retrieved from the API.

**Examples**

```
>>> from pynssp import *
>>>
>>> myProfile = create_profile()
>>> url = "http://httpbin.org/image/png"
>>> api_graph = get_api_graph(url, profile=myProfile)
```

pynssp.utils.**get_api_response**(*url*, *profile=None*)

Retrieve a response from an API using the provided profile.

**Parameters**

- **url** – A string representing the URL of the API endpoint.
- **profile** – An profile object of class *pynssp.core.credentials.Credentials* or *pynssp.core.token.Token*.

**Returns**

The response object returned by the API.

**Examples**

```
>>> from pynssp import *
>>>
>>> myProfile = create_profile()
>>> url = "http://httpbin.org/json"
>>> response = get_api_response(url, profile=myProfile)
```

pynssp.utils.**get_essence_data**(*url*, *start_date=None*, *end_date=None*, *profile=None*, ***kwargs*)

Retrieve data from the NSSP-ESSENCE API using the provided profile.

**Parameters**

- **url** – A string representing the URL of the NSSP-ESSENCE API endpoint.
- **start_date** – A string representing the start date of the data to retrieve.
- **end_date** – A string representing the end date of the data to retrieve.
- **profile** – An profile object of class *pynssp.core.credentials.Credentials* or *pynssp.core.token.Token*.
- **kwargs** – Additional arguments to be passed to the get_api_data function.

**Returns**

The data retrieved from the NSSP-ESSENCE API.

**Examples**

```
>>> from pynssp import *
>>>
>>> myProfile = create_profile()
>>> url = "https://essence2.syndromicsurveillance.org/
↪nssp_essence/api/timeSeries/graph?endDate=25Jun2022&
↪geography=&percentParam=noPercent&datasource=va_hosp&
↪startDate=25Jun2021&
↪medicalGroupingSystem=essencesyndromes&userId=3751&
↪aqtTarget=TimeSeries&ccddCategory=&
↪geographySystem=hospitalregion&detector=probrepswitch&
↪timeResolution=daily"
```

(continues on next page)

(continued from previous page)

```
>>> api_data = get_essence_data(url, profile=myProfile)
>>> api_data.info()
```

pynssp.utils.**webscrape_icd**(*icd_version='ICD10'*, *year=None*)

ICD Code Web Scraper

Function to web scrape ICD discharge diagnosis code sets from the CDC FTP server (for ICD-10) or CMS website (for ICD-9). If pulling ICD-10 codes, by default the function will search for the most recent year's code set publication by NCHS. Users can specify earlier publication years back to 2019 if needed. The ICD-9 option will only web scrape the most recent, final ICD-9 code set publication (2014) from the CMS website. This function will return an error message if the FTP server or CMS website is unresponsive or if a timeout of 60 seconds is reached. The result is a dataframe with 3 fields: code, description, and set (ICD version concatenated with year). Codes are standardized to upper case with punctuation and extra leading/tailing white space removed to enable successful joining.

**Parameters**

- **icd_version** – The version of ICD codes to retrieve. Default is "ICD10".
- **year** – The year for which to retrieve the ICD codes. If not provided, the current year will be used. (Default value = None)

**Returns**

A DataFrame containing the ICD codes and descriptions.

**Examples**

```
>>> # Example 1
>>> from pynssp import webscrape_icd
>>>
>>> icd9 = webscrape_icd(icd_version = "ICD9")
>>> icd9.head()
```

```
>>> # Example 2
>>> from pynssp import webscrape_icd
>>>
>>> icd10_2025 = webscrape_icd(icd_version="ICD10",␣
↪year=2025)
>>> icd10_2025.info()
```

```
>>> # Example 3
>>> from pynssp import webscrape_icd
>>>
```

(continues on next page)

(continued from previous page)

```
>>> icd10_2026 = webscrape_icd(icd_version="ICD10",␣
↪year=2026)
>>> icd10_2026.info()
```

### 6.1.5 Module contents

Top-level package for pynssp.

Chapter 7

# Release Notes

## 7.1 0.2.0 (2026-11-03)

### 7.1.1 Bug fixes

- The install issue related to package data has been fixed.
- The package info to document Python support up to 3.14 has been updated
- The `change_dates()` function has been upgraded to fix flip-flopping start and end date values.
- The EWMA detector has been updated to closely match the `Rnssp`'s implementation. In addition, the `alert_ewma()` function has been updated with a patch to prevent errors when detector is applied to a zero series.
- The Regression detector has been updated to closely match the `Rnssp`'s implementation.
- The Switch detector has been updated to closely match the `Rnssp`'s implementation.

### 7.1.2 New Features

#### Anomaly Detection

- The Farrington Temporal Detector algorithm `alert_farrington()` has been added for weekly time series of counts spanning multiple years.

#### Credentials Management

- A new abstract class `Auth` has been added.
- Both the `Credentials` and `Token` classes inherit the `Auth` class.
- A new `Apikey` class has been added for API services token use.

#### Classes

- The `Auth` class delineates methods Shared by the `Token`, `Credentials`, and `Apikey` Classes. The new `Apikey` class inherits the `Auth` class, and has the same methods as the `Credentials` and `Token` classes.

#### Utility Functions

- A new `create_apikey_profile()` function has been added as a wrapper to the `Apikey` class.

### 7.1.3 Other

- A section on the Farrington Temporal detector has been added to the Anomaly Detection vignette.
- A trend classification section has been added to the Anomaly Detection vignette.
- A new vignette entitled "Text Mining Example" has been added to the package documentation.
- The online documentation has a new theme.

## 7.2 0.1.0 (2023-06-22)

- First release on PyPI.

### 7.2.1 Features

#### Credentials Management

- `create_profile()` initializes a `Credentials` object from a username and password.
- `create_token_profile()` initializes a `Token` object from a REST API token.

#### REST API data pulls

- `get_api_response()` retrieves a response from an API service using a provided profile of type `Credentials` or `Token`. Used as a wrapper to the `.get_api_response()` method of a provided profile of type `Credentials` or `Token`.
- `get_api_data()` retrieves data (in JSON or CSV) from an API service using a provided profile of type `Credentials` or `Token`. Used as a wrapper to the `.get_api_data()` method of a provided profile of type `Credentials` or `Token`.
- `get_api_graph()` retrieves a graph from an API service using a provided profile of type `Credentials` or `Token`. Used as a wrapper to the `.get_api_graph()` method of a provided profile of type `Credentials` or `Token`.

- `get_essence_data()` retrieves data from the NSSP-ESSENCE API service using a provided profile of type `Credentials` or `Token`.

### Anomaly Detection and Trend Classification

- `alert_ewma()` implements the EWMA time series anomaly detection algorithm.
- `alert_regression()` implements the Multiple Adaptive Regression time series anomaly detection algorithm.
- `alert_switch()` implements the Regression/EWMA Switch time series anomaly detection algorithm
- `alert_nbinom()` implements the Negative Binomial Regression time series anomaly detection algorithm.
- `alert_serfling()` implements the original and modified Serfling method for time series anomaly detection.
- `classify_trend()` fits rolling binomial models to a daily time series of percentages or proportions to classify the overall trend.

### Added Data

- `load_simulated_ts()` loads a dataframe of simulated time series.
- `load_nssp_stopwords()` loads a dataframe of NSSP-curated stopwords.
- `get_scenario1()` loads a a subset of the simulated time series data ("scenario #1").
- `get_scenario2()` loads a a subset of the simulated time series data ("scenario #2").

### Utility Functions

- `change_dates()` modifies the start and end dates in a given URL to new dates, if provided.
- `webscrape_icd()` scrapes ICD Codes from the Web.

### Classes

- `Credentials` is an abstract representation of a profile object given a username and password strings.
- `Token` is an abstract representation of a profile object given a token string.
- `APIGraph` is an abstract representation of a graph object returns from an API service.
- `NSSPContainer` encapsulates a value or an object to store.

Chapter 8

# Credits

## 8.1 Development Lead

- Gbedegnon Roseric Azondekon

## 8.2 Contributors

- Andrew Manigault
- Alan Viars
- Huy Nguyen Dao
- Rakesh Pai

Chapter 9

# Contributing

## 9.1 Welcome!

Thank you for contributing to CDC's Open Source projects! If you have any questions or doubts, don't be afraid to send them our way. We appreciate all contributions, and we are looking forward to fostering an open, transparent, and collaborative environment.

Before contributing, we encourage you to also read or LICENSE, README, and code-of-conduct files, also found in this repository. If you have any inquiries or questions not answered by the content of this repository, feel free to contact us.

## 9.2 Public Domain

This project is in the public domain within the United States, and copyright and related rights in the work worldwide are waived through the CC0 1.0 Universal public domain dedication. All contributions to this project will be released under the CC0 dedication. By submitting a pull request you are agreeing to comply with this waiver of copyright interest.

## 9.3 Requesting Changes

Our pull request/merging process is designed to give the CDC Surveillance Team and other in our space an opportunity to consider and discuss any suggested changes. This policy affects all CDC spaces, both on-line and off, and all users are expected to abide by it.

### 9.3.1 Open an issue in the repository

If you don't have specific language to submit but would like to suggest a change or have something addressed, you can open an issue in this repository. Team members will respond to the issue as soon as possible.

### 9.3.2 Submit a pull request

If you would like to contribute, please submit a pull request. In order for us to merge a pull request, it must:

- Be at least seven days old. Pull requests may be held longer if necessary to give people the opportunity to assess it.
- Receive a +1 from a majority of team members associated with the request. If there is significant dissent between the team, a meeting will be held to discuss a plan of action for the pull request.

Chapter 10

# Disclaimer

Use of this service is limited only to **non-sensitive and publicly available data**. Users must not use, share, or store any kind of sensitive data like health status, provision or payment of healthcare, Personally Identifiable Information (PII) and/or Protected Health Information (PHI), etc. under **ANY** circumstance.

Administrators for this service reserve the right to moderate all information used, shared, or stored with this service at any time. Any user that cannot abide by this disclaimer and Code of Conduct may be subject to action, up to and including revoking access to services.

The material embodied in this software is provided to you "as-is" and without warranty of any kind, express, implied or otherwise, including without limitation, any warranty of fitness for a particular purpose. In no event shall the Centers for Disease Control and Prevention (CDC) or the United States (U.S.) government be liable to you or anyone else for any direct, special, incidental, indirect or consequential damages of any kind, or any damages whatsoever, including without limitation, loss of profit, loss of use, savings or revenue, or the claims of third parties, whether or not CDC or the U.S. government has been advised of the possibility of such loss, however caused and on any theory of liability, arising out of or in connection with the possession, use or performance of this software.

Chapter 11

# Creating a Culture of Innovation

We aspire to create a culture where people work joyfully, communicate openly about things that matter, and provide great services globally. We would like our team and communities (both government and private sector) to reflect on diversity of all kinds, not just the classes protected in law. Diversity fosters innovation. Diverse teams are creative teams. We need a diversity of perspective to create solutions for the challenges we face.

This is our code of conduct (adapted from 18F's Code of Conduct). We follow all Equal Employment Opportunity laws and we expect everyone we work with to adhere to the GSA Anti-harrasment Policy, even if they do not work for the Centers for Disease Control and Prevention or GSA. We expect every user to follow this code of conduct and the laws and policies mentioned above.

## 11.1 Be Empowering

Consider what you can do to encourage and support others. Make room for quieter voices to contribute. Offer support and enthusiasm for great ideas. Leverage the low cost of experimentation to support your colleagues' ideas, and take care to acknowledge the original source. Look for ways to contribute and collaborate, even in situations where you normally wouldn't. Share your knowledge and skills. Prioritize access for and input from those who are traditionally excluded from the civic process.

## 11.2 Rules of Behavior

- I understand that I must complete security awareness and records management training annually in order to comply with the latest security and records management policies.
- I understand that I must also follow the Rules of Behavior for use of HHS Information Resources
- I understand that I must not use, share, or store any kind of sensitive data (health status, provision or payment of healthcare, PII, etc.) under ANY circumstance.

- I will not knowingly conceal, falsify, or remove information.
- I understand that I can only use non-sensitive and/or publicly available data.
- I understand that all passwords I create to set up accounts need to comply with CDC's password policy.
- I understand that the stewards reserves the right to moderate all data at any time.

## 11.3 Boundaries

Create boundaries to your own behavior and consider how you can create a safe space that helps prevent unacceptable behavior by others. We can't list all instances of unacceptable behavior, but we can provide examples to help guide our community in thinking through how to respond when we experience these types of behavior, whether directed at ourselves or others.

If you are unsure if something is appropriate behavior, it probably is not. Each person we interact with can define where the line is for them. Impact matters more than intent. Ensuring that your behavior does not have a negative impact is your responsibility. Problems usually arise when we assume that our way of thinking or behavior is the norm for everyone.

### 11.3.1 Here are some examples of unacceptable behavior

- Negative or offensive remarks based on the protected classes as listed in the GSA Anti-harrasment Policy of race, religion, color, sex, national origin, age, disability, genetric information, sexual orientation, gender identity, parental status, maritual status, and political affiliation as well as gender expression, mental illness, socioeconomic status or backgrounds, neuro(a)typicality, physical appearance, body size, or clothing. Consider that calling attention to differences can feel alienating.
- Sustained disruption of meetings, talks, or discussions, including chatrooms.
- Patronizing language or behavior.
- Aggresive behavior, such as unconstructive criticism, providing correction that do not improve the conversation (sometimes referred to as "well actually's"), repeatedly interrupting or talking over someone else, feigning surprise at someone's lack of knowledge or awareness about a topic, or subtle prejudice.
- Referring to people in a way that misidentifies their gender and/or rejects the validity of their gender identity; for instance by using incorrect pronouns or forms of address (misgendering).
- Retaliating against anyone who files a formal complaint that someone has violated these codes or laws.

## 11.4 Background

CDC Scientific Clearance is the process of obtaining approvals by appropriate CDC officials before a CDC information product is released to the public or CDC's external public health partners. Information products that require formal clearance include print, electronic, or oral materials, that CDC employees author or co-author, whether published by CDC or outside CDC. CDC contractors developing content on behalf of CDC for the public or CDC's external public health partners are also required to put their content through the formal clearance process. The collaborative functions related to the projects include blogs, wikis, forums, bug tracking sites, source control and others deemed as necessary.

For those individuals within the CDC, adherence to the following policies are required:

- CDC "Clearance of Information Products Disseminated Outside CDC for Public Use"
- HHS "Ensuring the Quality of Information Disseminated by HHS agencies"

All collaborative materials will be controlled by the rules contained within this document. This will allow for the real-time collaboration opportunities among CDC employees, CDC contractors and CDC public health partners.

## 11.5 Credit

This code of conduct was mainly adapted from 18F's Code of Conduct and the CDC's Informatics Innovation Unit R&D Lab's code of conduct.

## 11.6 Relevant Legal Considerations

- Laws enforced by the Equal Employment Opportunity Commission
- Types of discrimination prohibited by law
- New and proposed regulations

Chapter 12

# Indices and tables

- genindex
- modindex
- search

# Python Module Index

## p

pynssp, 64
pynssp.core, 48
pynssp.core.constants, 46
pynssp.core.container, 46
pynssp.core.credentials, 47
pynssp.core.token, 48
pynssp.data, 57
pynssp.detectors, 57
pynssp.detectors.ewma, 48
pynssp.detectors.nbinom, 50
pynssp.detectors.regression, 52
pynssp.detectors.serfling, 53
pynssp.detectors.switch, 55
pynssp.detectors.trend, 56
pynssp.utils, 59

# Index


## A

adaptive_regression() (*in module pynssp.detectors.regression*), 52
alert_ewma() (*in module pynssp.detectors.ewma*), 48
alert_nbinom() (*in module pynssp.detectors.nbinom*), 50
alert_regression() (*in module pynssp.detectors.regression*), 52
alert_serfling() (*in module pynssp.detectors.serfling*), 53
alert_switch() (*in module pynssp.detectors.switch*), 55
APIGraph (*class in pynssp.core.container*), 46

## C

change_dates() (*in module pynssp.utils*), 59
classify_trend() (*in module pynssp.detectors.trend*), 56
create_apikey_profile() (*in module pynssp.utils*), 59
create_profile() (*in module pynssp.utils*), 60
create_token_profile() (*in module pynssp.utils*), 60
Credentials (*class in pynssp.core.credentials*), 47

## E

ewma_loop() (*in module pynssp.detectors.ewma*), 50

## G

get_api_data() (*in module pynssp.utils*), 60
get_api_graph() (*in module pynssp.utils*), 61
get_api_response() (*in module pynssp.utils*), 61
get_api_response() (*pynssp.core.credentials.Credentials method*), 47
get_api_response() (*pynssp.core.token.Token method*), 48
get_essence_data() (*in module pynssp.utils*), 62
get_scenario1() (*in module pynssp.data*), 57
get_scenario2() (*in module pynssp.data*), 58
get_trends() (*in module pynssp.detectors.trend*), 57

## L

load_nssp_stopwords() (*in module pynssp.data*), 58
load_simulated_ts() (*in module pynssp.data*), 58

## M

module
    pynssp, 64
    pynssp.core, 48
    pynssp.core.constants, 46
    pynssp.core.container, 46
    pynssp.core.credentials, 47
    pynssp.core.token, 48
    pynssp.data, 57
    pynssp.detectors, 57
    pynssp.detectors.ewma, 48
    pynssp.detectors.nbinom, 50
    pynssp.detectors.regression, 52

pynssp.detectors.serfling, 53
pynssp.detectors.switch, 55
pynssp.detectors.trend, 56
pynssp.utils, 59

## N

nb_model() (*in module pynssp.detectors.nbinom*), 51
NSSPContainer (*class in pynssp.core.container*), 46

## P

plot() (*pynssp.core.container.APIGraph method*), 46
pynssp
    module, 64
pynssp.core
    module, 48
pynssp.core.constants
    module, 46
pynssp.core.container
    module, 46
pynssp.core.credentials
    module, 47
pynssp.core.token
    module, 48
pynssp.data
    module, 57
pynssp.detectors
    module, 57
pynssp.detectors.ewma
    module, 48
pynssp.detectors.nbinom
    module, 50
pynssp.detectors.regression
    module, 52
pynssp.detectors.serfling
    module, 53
pynssp.detectors.switch
    module, 55
pynssp.detectors.trend
    module, 56
pynssp.utils
    module, 59

## S

serfling_model() (*in module pynssp.detectors.serfling*), 54
show() (*pynssp.core.container.APIGraph method*), 46

## T

Token (*class in pynssp.core.token*), 48

## W

webscrape_icd() (*in module pynssp.utils*), 63